



Chaotic map-coded metaheuristics for metameric variable-length problems

Haichuan Yang^{1,2} · Haotian Li³ · Yifei Yang⁴ · Naoya Chiba^{2,5} · Shingo Kagami² · Koichi Hashimoto² · Yuichi Nagata¹

Received: 28 November 2024 / Revised: 9 May 2025 / Accepted: 11 May 2025 /

Published online: 4 June 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Optimizing systems with an unknown number of critical homogenous components presents a significant challenge in many real-world applications. Traditional metaheuristic approaches often require predefined component counts, leading to suboptimal solutions when this number is uncertain. To address this, we propose a chaotic map-encoded metaheuristic framework that dynamically adjusts the number of components during optimization. This approach is applied to two complex problems: dendritic neuron model (DNM) optimization, where the goal is to determine the optimal number of dendritic branches for improved learning performance, and wind farm layout optimization (WFLOP), which seeks to optimize the placement and number of wind turbines to maximize energy output while minimizing wake effects. Experimental results show that this approach outperforms variable-length genetic algorithms in DNM optimization and demonstrates competitive performance in WFLOP. These findings highlight the potential of chaotic maps to improve metaheuristic efficiency in variable-length optimization problems.

Keywords Variable length problem · Metaheuristics · Chaotic map · Indirect encoding

Abbreviations

CGLPSO	Improved versions of GLPSO based on chaotic map
CSHADE	Improved versions of SHADE based on chaotic map
CSIS	Improved versions of SIS based on chaotic map
DNA	Populations involved in the iterative process of the metaheuristics
DNM	Dendritic neuron model
GLPSO	Genetic learning particle swarm optimization
PWLCM	Piecewise linear chaotic map

Extended author information available on the last page of the article

PSO	Particle swarm optimization
RNA	Populations involved in evaluation after chaotic transcription of DNA
SHADE	Success-history based parameter adaptation for differential evolution
SIS	Spatial information sampling algorithm
VGA	Variable-length genetic algorithms
WFLOP	Wind farm layout optimization problem

1 Introduction

Optimization plays a pivotal role in scientific and engineering problem-solving, enabling the development and application of innovative techniques to tackle complex challenges [1]. Particularly, metaheuristics have emerged as a prominent method for achieving practical solutions in scenarios where ideal solutions are elusive, such as in NP-complete problems [2, 3]. Among early population-based metaheuristics, the Genetic Algorithm (GA) [4] was foundational and inspiring successors like Differential Evolution (DE) [5], which improved mutation by leveraging individual differences. SHADE [6] further improved DE by adaptively linking offspring to optimal parents and preserving superior parameter settings for future generations. This evolution mirrors the development of social organization, leading to knowledge transmission. The DE family of algorithms has been highly successful, often topping the IEEE CEC competition [7, 8]. Additionally, the field of metaheuristics has branched into swarm intelligence, with algorithms like Ant Colony Optimization [9], Particle Swarm Optimization (PSO) [10], and newer developments like Genetic Learning Particle Swarm Optimization (GLPSO) [11] and Spatial Information Sampling Algorithm (SIS) [12]. These algorithms simulate decentralized, self-organized behaviors observed in nature, from cooperative predation to gravitational interactions among celestial bodies. Based on the aforementioned algorithm, research on the improvement of metaheuristics has also been continuously emerging. In study [13], a metaheuristic operator integration method based on Unstructured Evolutive Game Theory was proposed to regulate and integrate the strengths of different algorithms. The study [14] focuses on the independent strategies of individuals within the algorithmic population, promoting search diversity and enhancing solution efficiency in complex search spaces. The study [15] proposed a metaheuristic assisted by a Radial Basis Function Neural Network, effectively balancing exploration and exploitation in the optimization process.

In conventional optimization techniques, solutions are often represented through vectors of constant dimension. This fixed-dimension approach underpins much of the optimality theory, premised on well-defined gradients in a static variable space. However, numerous real-world problems defy this rigid structure, requiring a variable number of elements for their resolution. Typical optimization methods adapt by predetermining variable count, yet this can yield suboptimal outcomes if the selected count deviates from the ideal. Iteratively adjusting this number is theoretically possible but proves inefficient, particularly when potential variations are extensive [16]. The scope of this investigation is confined to problems characterized by solutions

composed of homogeneous components. These include but are not limited to, applications in wireless sensor networks [17], wind farm layouts [18], and the dendritic neuron model structure [19]. In these contexts, the solution to this problem is valid regardless of the number of components (such as the number of sensors, turbines, or dendritic branches). Instead, the focus shifts to finding the optimal quantity of these components, a figure that is typically indeterminate at the outset. Each component is defined by an identical set of parameters (e.g., coordinates or angles), though the specific values may vary across components. This kind of problem is defined as a metameric variable-length problem [16]. In biology, the term metameric is used to describe anatomies that are composed of structurally similar, but not necessarily identical, segments. Examples include a vertebrate spine or the segments of an arthropod [20]. Figure 1 presents a typical example of the problem under investigation. The image illustrates the training process of an artificial neuron model, where the black dots represent weights. The one-dimensional and two-dimensional solution spaces on the left and right, respectively, correspond to the solution spaces when the neuron model has 1 and 2 weights. As the number of weights increases, the dimensionality of the solution space also increases. During the algorithm's iteration process, the solution spaces on the left and right frequently alternate.

In response to this challenge, variable-length metaheuristics have been developed. In metameric representation, various methodologies can be employed for solution encoding. Predominantly, there are two principal approaches: direct and indirect encoding [21]. As defined by [22], direct encoding maintains a congruence between the genotypic space (that is, the metavariables) and the phenotypic space (namely, the solution after evaluation). Conversely, indirect encoding necessitates one or more intermediary processes in the transition from genotype to phenotype, potentially facilitating the emergence of more intricate phenotypes from comparatively simpler genotypes. In direct encoding methods, the specific encoding techniques vary according to the problem at hand. However, the core methodology typically involves two key aspects: Firstly, the insertion and removal (or hiding) of certain loci in genes during the mutation process [23, 24]. Secondly, the exchange of gene segments of varying lengths during the crossover process of individuals [25, 26]. The aforementioned methods are mostly based on the mainstream algorithm of GA. Additionally, there are other instances where fixed-length metaheuristics

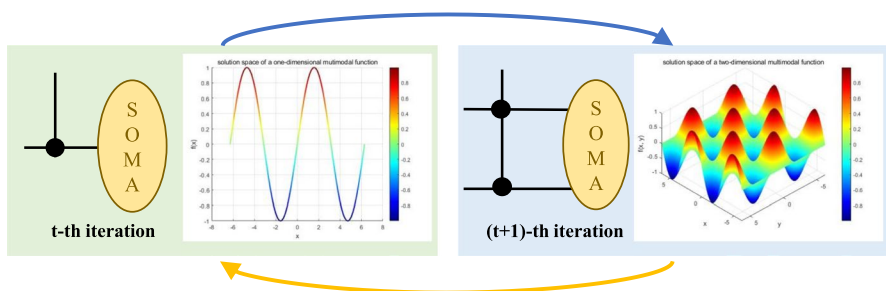


Fig. 1 Dimensional changes in the solution space during the algorithm's iteration process of solving the metameric variable-length problem, exemplified with a maximum of two dimensions

have been adapted into variable-length metaheuristics [27–32]. Research in [21] has highlighted that while the technical details employed in these operations differ, there is considerable overlap. Furthermore, they have also identified inherent challenges with direct encoding methods: Altering the length of a solution, whether through crossover or mutation, generally has a negative impact on the fitness of the solution. As the solution length varies, the entire fitness landscape shifts accordingly. This change means that the original building blocks in the genotype may no longer align optimally with the new length of the solution. This adverse effect can lead to such solutions being rapidly eliminated by standard selection processes.

Compared to direct encoding, current indirect encoding methods are relatively more complex but demonstrate superior performance in handling variable-length problems with numerous components [21]. An influential study in indirect encoding is the Compositional Pattern Producing Networks (CPPNs) introduced in 2007 [33]. The core idea of this study is to describe the structural relationships of the developmental process directly through function composition. This approach is similar to neural networks but utilizes different mathematical functions (e.g., Gaussian and periodic functions) to generate patterns. It also enables continuous complexification during evolution, leading to increasingly complex structures and regularities. CPPNs can evolve characteristics akin to natural development, such as symmetry, reuse, repetition with variation, and preservation of regularities, validating it as an effective abstraction of indirect encoding. CPPNs have spurred a surge in the study of indirect encoding within the field of neural networks, directly influencing the development of neuroevolution in designing neural networks [34]. Given the substantial evidence provided by the aforementioned research on the significance of indirect encoding, we have attempted to propose a concise and efficient method of indirect encoding to enhance the diversity of algorithms.

Our research direction is inspired by the inherent chaos in biological coding systems. The “central dogma of molecular biology,” as stated in [35], posits that all biological information originates from Deoxyribonucleic Acid (DNA), is relayed through Ribonucleic Acid (RNA), and ultimately expressed as proteins. However, an increasing body of evidence contradicts this linear pathway. Environmental factors, including metabolic influences, post-translational effects, and inheritable epigenetic mechanisms, lead to diverse gene expressions, challenging the establishment of straightforward relationships between gene expression and cellular phenotypes [36]. Consequently, it’s reductive to view specific phenotypic traits as mere outcomes of independently acting genes, especially in cases of single allelic variations. Reference [37] proposed that chaotic systems could play a significant role in gene expression and translation dynamics. Echoing this, Heltberg et al. [38] emphasized the advantageous heterogeneity chaos introduces in populations. Motivated by these biological findings, our approach integrates a chaotic map into the gene expression mechanism in our metaheuristics model. In our methodology, individuals are divided into two layers: DNA and RNA. The DNA layer genes serve as the initial conditions for the chaotic map, all uniformly dimensioned. The first gene of each individual dictates the dimensions in the RNA layer. This chaotic map then transforms DNA genes into RNA individuals with variable dimensions. The fitness of these RNA individuals is evaluated and impacts the DNA individuals for evolutionary iterations

within the population. We term this innovative process “Chaotic Transcription.” It is important to emphasize that other metaheuristic methods based on chaotic maps typically use sequences generated by chaotic maps as thresholds or parameters to introduce additional randomness into the algorithm. For example, chaotic maps are often incorporated as step-size parameters during the search process to enhance exploration capability and increase the likelihood of escaping local optima [39]. In contrast, our study fundamentally differs from these approaches, as it represents the first application of chaotic maps in the encoding process of metaheuristic algorithms.

The main contributions of this study can be summarized as:

- The paper proposes an original method inspired by biological chaos, integrating a chaotic map into the gene expression mechanism of the metaheuristic model. This approach, simulating the dynamics of gene expression and translation, introduces heterogeneity and adaptability into the population, potentially enhancing the efficacy and diversity of solutions for metamer variable-length problems.
- Through experiments conducted on dendritic neuron model optimization problems (continuous problem) and wind farm layout optimization problems (discrete problem), we have arrived at the following conclusions: (1) The optimization results for the dendritic neuron model indicate that metaheuristic algorithms encoded with chaotic maps surpass mainstream variable-length genetic algorithms in performance. (2) The optimization results for wind farm layouts indicate that while the metaheuristic algorithm encoded with chaotic maps does not show a significant advantage over mainstream variable-length genetic algorithms when dealing with metamer variable-length problems, it unexpectedly performs significantly better than fixed-length metaheuristic algorithms when handling fixed-length problems.

This paper builds upon our conference paper, which has been accepted for presentation at The IEEE World Congress on Computational Intelligence [40]. In this extended version, we have significantly expanded our study by incorporating additional experiments (wind farm layout optimization problems), in-depth analysis, and new theoretical insights that were not included in the original conference submission. The rest of the paper is organized as follows: Related works are introduced in Sect. 2. Section 3 is the simulation models of optimized problems. Metaheuristics with chaotic maps are introduced in Sect. 4. The experimental results are introduced in Sect. 5. Section 6 contains the discussion and conclusion.

2 Related works

2.1 Dendritic neuron model optimization problems

In 1943, reference [41] introduced a seminal mathematical model depicting neuronal activities, laying the groundwork for Artificial Neural Networks (ANNs). The evolution of computer technology has catalyzed the continuous advancement

of ANNs, leading to the development of variants like Convolutional Neural Networks [42], and Recurrent Neural Networks [43]. A significant milestone was the introduction of the Transformer model by Vaswani et al. in 2017 [44]. This model has inspired subsequent innovations including BERT [45] and GPT [46]. Some research has also explored emulating biological neural structures to enhance performance. Building upon the McCulloch and Pitts neuron model, some prominent developments include the Spiking Neuron Model [47] and the Dendritic Neuron Model (DNM) [19]. Spiking neurons replicate the temporal dynamics of biological neurons, generating spike-based outputs, which are instrumental in constructing time-coded neural networks for efficient sequential data processing. Conversely, DNM models focus on the spatial characteristics of neurons, incorporating sigmoid functions and multiplicative operations for synaptic and dendritic interactions, respectively. Our study leverages the DNM to explore the application of metaheuristics in the evolution of neuronal models. Addressing the training complexities of neuronal models, often akin to NP-hard problems [48], metaheuristics have emerged as a potent approach. Applying metaheuristics to train DNM has yielded promising results in domains like financial time series prediction [49] and photovoltaic power forecasting [50]. However, training DNMs involves tuning multiple hyperparameters, including k , q , and m , representing distance, firing, and dendritic branch parameters, respectively. Previous studies segmented k and q into five adjustable parameters each, while m involved up to 20 variations. From an orthogonal experimental design perspective, this requires over 500 parameter experiments, posing a significant challenge. Thus, introducing methods for adaptive parameter optimization is crucial. Our previous work successfully integrated k and q into the iterative process of metaheuristics, enabling the self-evolution of parameters [51]. Nonetheless, incorporating m remains challenging due to the resultant change in computational dimensions, categorizing the problem as a metameric variable-length problem, as shown in Fig. 1.

2.2 Wind farm layout optimization problems

Renewable energy, prominently featuring wind energy, plays a vital role in socioeconomic advancement and economic development. Wind energy has gained significant attention in academic and policy circles [52–54], especially with the advent of artificial intelligence technologies that have been integrated into wind energy conversion systems [55]. This growth in wind energy technology underscores the need for accurate data on spatial costs and potential distributions. A review by Russell McKenna et al. divides wind energy assessment into four broad categories: geographic and technical onshore wind potentials, economic potentials, and feasible onshore wind potentials [56]. However, this review also points out the challenges in detailed wind farm layout analysis on larger geographical scales. Harnessing wind energy is most efficiently done through wind farms, which consist of numerous wind turbines placed strategically to maximize wind power potential. However, these farms encounter a dilemma known as the wake effect, where downstream turbines suffer reduced power generation due to upstream turbines [57]. Wind Farm Layout

Optimization Problems (WFLOP) aim to enhance power generation efficiency by minimizing these wake effect losses. The complexity of WFLOP is significant: a wind farm with 30 turbines can present more than 10^{44} layout solutions [58]. This complexity is amplified by the “curse of dimensionality” as the number of turbines increases [59]. Due to the non-convex and NP-hard nature of WFLOP, traditional resolution techniques like linear programming and mixed-integer programming fall short [60]. Therefore, metaheuristics have been introduced as a solution for WFLOP. In WFLOP, GAs [61, 62] and their enhanced versions [63, 64] are widely used, as noted in [65]. Additionally, other algorithms like PSO [66], ACO [67], and multi-objective evolutionary algorithms [68] also being popular. Our recent studies on WFLOPs indicate that metaheuristics with Poisson-distributed population networks excel in certain wind farm models [69]. While the above fixed-length metaheuristics have demonstrated potential in addressing the WFLOP [70], their effectiveness is often limited by a prerequisite: the number of turbines must be predetermined. This limitation arises from the dependency of these algorithms on matrix operations during iterative processes. Any alteration in turbine count changes the dimensionality of the solution space, impeding effective iteration.

3 Simulation models of optimized problems

3.1 Dendritic neuron model

The model segregates a dendritic neuron into four distinct layers: synaptic, dendrite, membrane, and soma, each layer processing signals in a sequential manner from input to output.

Synaptic layer The synaptic layer processes signals from axon terminals to dendrites. This layer’s receptors uptake ions, with potentials varying based on synaptic states—either inhibitory or excitatory. The inputs $[U_1, \dots, U_\alpha, \dots, U_I]$ ($\alpha = 1, 2, \dots, I$) represent external stimuli to the presynaptic axon terminal, characterized by a sigmoid function:

$$Q_{\alpha\beta} = \frac{1}{1 + e^{-k(z_{\alpha\beta}x_\alpha - p_{\alpha\beta})}}. \quad (1)$$

Here, $z_{\alpha\beta}$ indicates synaptic weight, and $p_{\alpha\beta}$ is the threshold parameter, both influencing DNM’s dendritic and axonal morphology and subject to training. β ($\beta = 1, 2, \dots, m_i$) denotes the index of the dendrite branch. β is the same as j in Eq. (7). The parameter k denotes the distance factor, and $Q_{\alpha\beta}$ quantifies the post-dendritic cell’s response to the presynaptic input.

Optimizing $z_{\alpha\beta}$ and $p_{\alpha\beta}$ emulates synaptic plasticity, as depicted in Fig. 2. The synaptic plasticity is emulated by approximating connections into four states, each biologically representing the neuron’s morphology during morphological transitions.

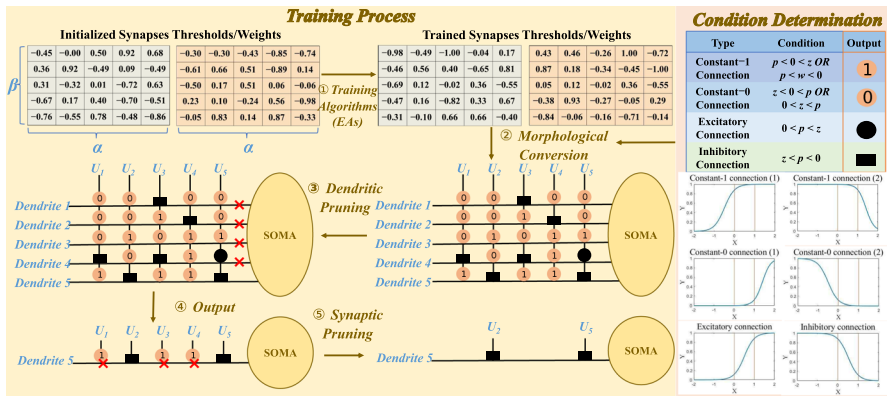


Fig. 2 The DNM training process encompasses five steps: algorithmic training, morphological conversion based on connection states, and both dendritic and synaptic pruning with their respective outputs. Pruning aids in eliminating redundant dendrites and synapses and inferring their connection dynamics. The red crosses in the figure don't signify a special pruning by the algorithm, but indicate that some Q values in a Dendrite are near 0, making Y almost 0, thus allowing the ignoring of this Dendrite. Similarly, the Q values near 1 can also be ignored (Color figure online)

1. Constant 1 Connection: Postsynaptic potential remains close to 1 with inputs between 0 and 1 when either $p_{\alpha\beta} < 0 < z_{\alpha\beta}$ or $p_{\alpha\beta} < z_{\alpha\beta} < 0$.
2. Constant 0 Connection: Potential stays near 0 with varying inputs from 0 to 1, if $z_{\alpha\beta} < 0 < p_{\alpha\beta}$ or $0 < z_{\alpha\beta} < p_{\alpha\beta}$.
3. Excitatory Connection: Potential is directly proportional to input (ranging 0 to 1) given $0 < p_{\alpha\beta} < z_{\alpha\beta}$.
4. Inhibitory Connection: Potential is inversely proportional to input (0 to 1 range) if $z_{\alpha\beta} < p_{\alpha\beta} < 0$.

Dendritic layer In cases where the dendritic inputs are binary (1 or 0), the multiplication operator functions analogously to the logical AND operation. This concept can be succinctly described as follows:

$$Y_{\beta} = \prod_{\alpha=1}^I Q_{\alpha\beta}. \quad (2)$$

Membrane layer The membrane layer aggregates signals from the dendritic layer. It applies a summation operator akin to a logical OR operation, relaying the combined signals to the soma:

$$V = \sum_{\beta=1}^{m_i} Y_{\beta}. \quad (3)$$

Soma layer Finally, the soma employs a sigmoid function for the membrane layer V to determine neuronal firing based on overall output. The firing parameter q ranges from 0 to 1, setting the soma threshold:

$$W = \frac{1}{1 + e^{-k(V-q)}}. \quad (4)$$

In this study, we choose DNM as the subject of neuroevolution research because metaheuristic algorithms can optimize its thresholds $p_{\alpha\beta}$ and weights $z_{\alpha\beta}$, serving as the model parameters. Previous studies have shown that DNMs trained with metaheuristic algorithms can sometimes outperform those trained with backpropagation in specific real-world applications [71]. However, a well-known limitation of metaheuristic algorithms is their susceptibility to the curse of dimensionality [72]. As the number of model parameters β increases, these algorithms face exponential growth in search complexity, making them impractical for large-scale modern deep learning architectures.

Due to this limitation, metaheuristics have been predominantly applied to architecture and hyperparameter optimization rather than direct parameter tuning in deep neural networks. Our chaotic transcription technique may help advance research on this scalability issue. It uses the properties of chaotic maps, where a system's evolution depends only on its initial values, to compress high-dimensional parameters into a lower-dimensional space. Unlike conventional dimensionality reduction techniques, chaotic transcription can extend to high-dimensional spaces, potentially making metaheuristic parameter optimization feasible in complex artificial neural network settings. This capability provides an alternative potential pathway for exploring efficient parameter combinations in large-scale, high-dimensional solution spaces.

3.2 Wind farm layout model

There are typically five sub-models within wind farm layout problem models: wind condition model, wind farm land model, wake effect model, wind turbine model, and objective function model [69]. In our study, we modeled four specific wind scenarios, labeled as P_1 , P_2 , P_3 , and P_4 , each representing different wind patterns and speeds. The first scenario, P_1 , features a unidirectional wind flow at $\theta = 0$. The second scenario, P_2 , simulates wind coming from four directions: 0 , $\frac{\pi}{2}$, π , and $\frac{3\pi}{2}$, with an equal probability of 25% for each direction. In the third scenario, P_3 , the wind is modeled to come from six directions: 0 , $\frac{\pi}{3}$, $\frac{2\pi}{3}$, π , $\frac{4\pi}{3}$, and $\frac{5\pi}{3}$. The probabilities for these directions are set at 20%, 30%, 20%, 10%, 10%, and 10% respectively. Finally, the fourth scenario, P_4 , incorporates a more complex model with twelve wind directions: 0 , $\frac{\pi}{6}$, $\frac{\pi}{3}$, $\frac{\pi}{2}$, $\frac{2\pi}{3}$, $\frac{5\pi}{6}$, π , $\frac{7\pi}{6}$, $\frac{4\pi}{3}$, $\frac{3\pi}{2}$, $\frac{5\pi}{3}$, and $\frac{11\pi}{6}$, each having an equal occurrence probability of $\frac{1}{12}$. This scenario also varies wind speeds among 13 m/s, 10 m/s, and 7 m/s, with corresponding probabilities of 70%, 20%, and 10% respectively. Figure 3 displays the 12 wind farm layouts with constraints imposed

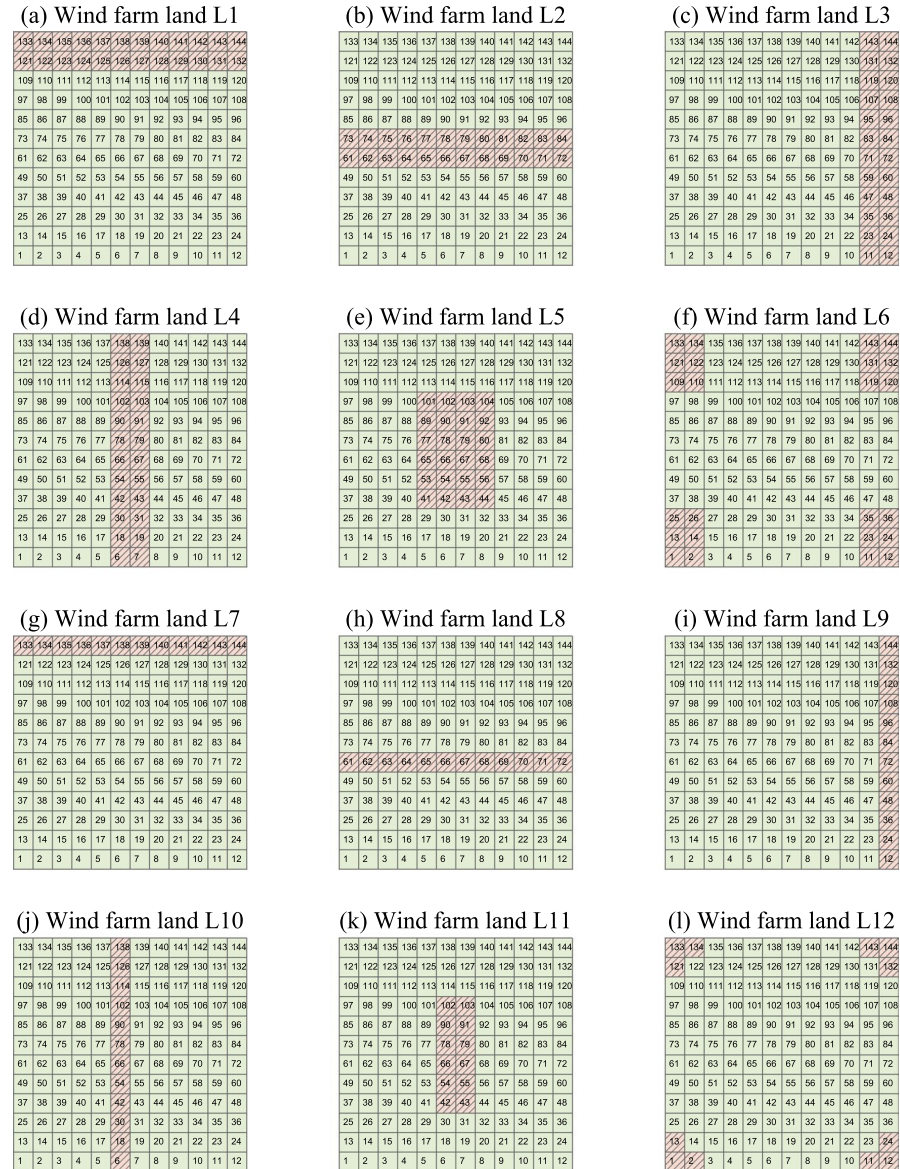


Fig. 3 Examples of various wind farm locations that are subject to landowner restrictions

by landowners, and the shaded areas indicate regions where turbines cannot be installed.

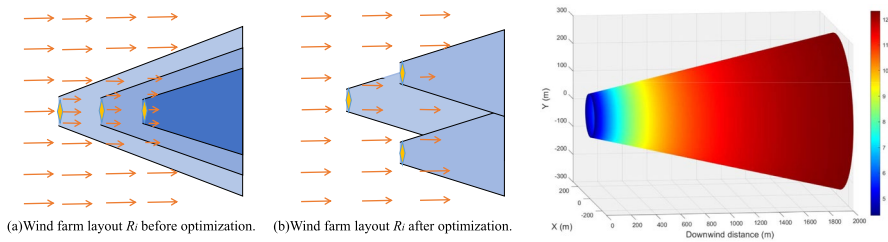


Fig. 4 Process and objectives of wind farm layout optimization

Optimizing power production in a wind farm, given a fixed wind distribution profile, requires strategic placement of each wind turbine. As illustrated in Fig. 4, which offers a top-down view of the wind farm layout, the wind turbines are represented by yellow rhombuses, the blue trapezoids indicate areas impacted by the wake effect, and the orange arrows depict the wind direction. The optimization challenge involves transitioning the wind farm layout from the suboptimal state shown in Fig. 4a to the ideal state in Fig. 4b. The objective function is defined as:

$$E(m_i, R_i) = \sum_{j=1}^{m_i} \sum_{\theta, v} p(\theta, v) o_j(\theta, v, R_i). \quad (5)$$

The function $E(m_i, R_i)$ represents the total power output of the m_i turbines arranged according to layout R_i . In metaheuristic terms, R_i is considered the algorithm's candidate solution. $p(\theta, v)$ indicating the probability distribution for these wind parameters, θ is the direction of the wind and v is the wind speed. The term $o_j(\theta, v, R_i)$ refers to the power output of the j -th wind turbine within layout R_i under specific wind conditions θ and v . The effectiveness of each layout is measured by the conversion efficiency η :

$$\eta = \frac{E(m_i, R_i)}{m_i \sum_{\theta, v} p(\theta, v) r(v)}, \quad (6)$$

where $r(v)$ represents the rated power output of a wind turbine at wind speed v , not accounting for the wake effect. The conversion efficiency η is used as a benchmark to compare different layouts, with a higher η indicating a more efficient layout. In this study, we choose WFLOP because it is a representative problem in variable-length optimization [21].

4 Metaheuristics with chaotic maps

Metaheuristic algorithms typically involve four fundamental operations: initialization, search, evaluation, and selection. The initialization phase sets initial parameters and populates the solution space with random individuals. Consider

a set of n individuals randomly initialized in a d -dimensional space, denoted as $X_i = [x_{i,1}, x_{i,2}, \dots, x_{i,j}, \dots, x_{i,d}]$. Here, i represents the individual's index and j denotes the dimension's index, with i ranging from 1 to n and j from 1 to d . The fitness of each individual is assessed using the function $f(X_i)$.

Research [73] highlights certain recurring patterns or characteristics in the search operations of metaheuristics, expressed as:

$$O_{i,j} = X_{r_1,j}^t + S\left(X_{r_2,j}^t, X_{r_3,j}^t, \dots, X_{r_n,j}^t\right). \quad (7)$$

In this equation, t indicates the iteration number, O represents the offspring population in transition, and X^t is the current parent population. The indexes $(r_1, r_2, \dots, r_n)$ identify individuals in the parent population, which vary based on the specific algorithm. Following the search operation, the metaheuristics evaluate the fitness of all individuals in O . Utilizing a greedy selection strategy, each O_i is compared with its counterpart in the original population X^t . Those with superior performance are selected for the next generation. This process is described by:

$$X_b^{t+1} = \begin{cases} O_i, & \text{if } f(O_i) > f(X_b^t), \\ X_b^t, & \text{otherwise.} \end{cases} \quad (8)$$

This formulation is for optimization towards the maximum value. In algorithms like SHADE and some versions of GA, $b = i$ and they are defined within the range $[1, 2, \dots, n]$. In contrast, algorithms like GLPSO, SIS, and some versions of GA define b as the index of the top-performing individual.

In this study, metaheuristics with chaotic maps have two parallel populations: DNA and RNA. Individuals in DNA are represented as $D_i = [m_i, z_{i,1}, y_i]$, where m_i is the number of dimensions in RNA, $z_{i,1}$ and y_i are the initial values and the control parameters in the Piecewise Linear Chaotic Map (PWLCM) [74], respectively. The use of the PWLCM in this study is due to its ability to uniformly produce values between 0 and 1, offering universality. m_i will determine the number of turbines (or dendritic branches) in a layout plan (or DNM structure) D_i , and $z_{i,1}$ and y_i are used to generate the specific locations of the turbines (or the specific structure of the DNM). The matrix DNA is shown in Eq. (9).

$$DNA = \begin{bmatrix} D_1 \\ D_2 \\ \vdots \\ D_i \\ \vdots \\ D_n \end{bmatrix} = \begin{bmatrix} m_1, z_{1,1}, y_1 \\ m_2, z_{2,1}, y_2 \\ \vdots \\ m_i, z_{i,1}, y_i \\ \vdots \\ m_n, z_{n,1}, y_n \end{bmatrix} \quad (9)$$

Note (1) The number of dimensions of the individuals in the DNA is consistent, and all are 3-dimensional. These individuals can only be used for search and selection operations during the iterative process of metaheuristics and cannot participate in the evaluation operation $f(\cdot)$

Algorithm 1 Pseudocode of metaheuristics with chaotic map

```

1: /* Initialization operation */
2: Randomly initialize  $n$  individuals of DNA.
3: Initialize the parameters.
4: for  $i = 1 : n$  do
5:   Transcribe  $D_i$  in DNA to  $R_i$  in RNA using Eq. (10).
6:   Evaluate the fitness value of  $R_i$  using  $f(R_i)$ .
7: end for
8: while Terminal Condition is not met do
9:   /* Search operation */
10:  for  $i = 1 : n$  do
11:    for  $j = 1 : d$  do
12:      Update the  $j$ -th dimension of individual  $i$  using Eq. (12).
13:    end for
14:  end for
15:  Perform boundary detection to ensure feasibility.
16:  for  $i = 1 : n$  do
17:    Transcribe  $D_i$  in DNA to  $R_i$  in RNA using Eq. (10).
18:    Evaluate the fitness value of  $R_i$  using  $f(R_i)$ .
19:  end for
20:  /* Selection operation */
21:  for  $i = 1 : n$  do
22:    Select individuals for the next generation using Eq. (13).
23:  end for
24: end while

```

As a kind of simple dynamic model, PWLCM can be regarded as the most simple map used in image encryption [75]. It has been known that PWLCM is ergodic and has a uniform invariant density function on its definition intervals [76]. The mathematical model of PWLCM can be described as follows:

$$z_{i,(j+1)} = \begin{cases} z_{i,j}/y_i, & z_{i,j} \in (0, y_i) \\ (1 - z_{i,j})/(1 - y_i), & z_{i,j} \in [y_i, 1) \end{cases} \quad (10)$$

where $z_{i,j}$ denotes the state variable, which is located in the interval $(0, 1)$. When y_i is taken in the interval $(0, 1)$, the map becomes chaotic. j represents the index

of z_i in the map, $j = [1, 2, \dots, m_i]$. After transcription of the information in DNA by PWLCM, RNA is generated. Individuals in RNA are represented as $R_i = [z_{i,1}, z_{i,2}, \dots, z_{i,j}, \dots, z_{i,m_i}]$. The set RNA is shown in Eq. (11).

$$RNA = \begin{Bmatrix} R_1 \\ R_2 \\ \vdots \\ R_i \\ \vdots \\ R_n \end{Bmatrix} = \begin{Bmatrix} z_{1,1}, z_{1,2}, \dots, z_{1,j}, \dots, z_{1,m_1} \\ z_{2,1}, z_{2,2}, \dots, z_{2,j}, \dots, z_{2,m_2} \\ \vdots \\ z_{i,1}, z_{i,2}, \dots, z_{i,j}, \dots, z_{i,m_i} \\ \vdots \\ z_{n,1}, z_{n,2}, \dots, z_{n,j}, \dots, z_{n,m_n} \end{Bmatrix} \quad (11)$$

Note (2) Because of differences in m_i , individuals in RNA have different numbers of dimensions. These individuals can only be used in the evaluation process $f(R_i)$ and cannot participate in the search and selection operations in the iterative process of metaheuristics.

After chaotic transcription, Eqs. (7), (8) are updated to Eqs. (12), (13):

$$O_{ij} = D_{r_{1j}}^t + S(D_{r_{2j}}^t, D_{r_{3j}}^t, \dots, D_{r_{nj}}^t) \quad (12)$$

$$D_b^{t+1} = \begin{cases} O_i, & \text{if } f(R_i) > f(R_b^t) \\ D_b^t, & \text{if otherwise} \end{cases} \quad (13)$$

The O in the formula is DNA. The transcription process from DNA to RNA is shown in Fig. 5, and there are 8 individuals in the figure. The matrix on the left side of the figure is DNA, each row represents an individual, and each column represents a gene under the same dimension. The irregular data set on the right side of the figure is RNA, with each row representing an individual whose color corresponds to

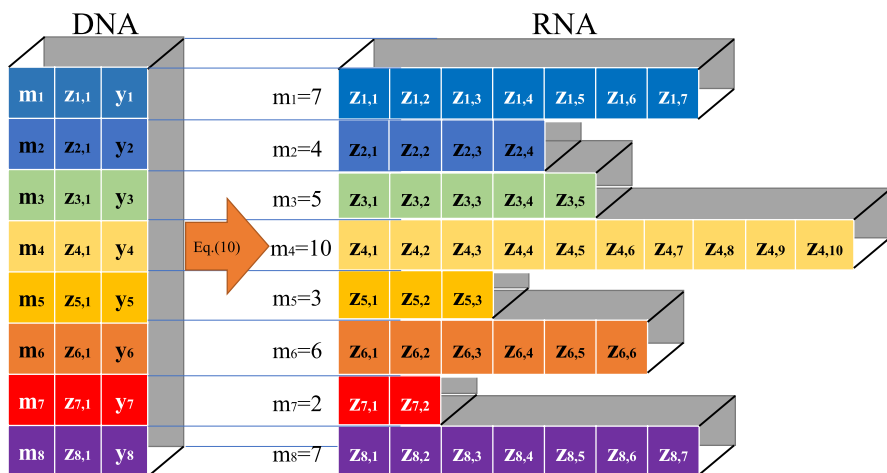


Fig. 5 The transcription process from DNA to RNA

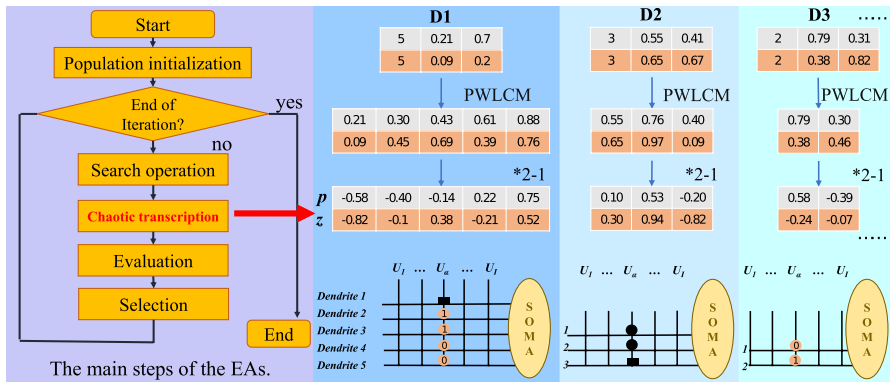


Fig. 6 The execution step of chaotic transcription in DNM optimization

the individual in the DNA. The pseudo-code for the improved metaheuristics using chaotic maps is shown in Algorithm 1.

To further illustrate the role of chaotic transcription, we provide examples in DNM optimization and WFLOP. Figure 6 is the execution step of chaotic transcription in DNM optimization. This illustration selectively represents the dendritic states corresponding to a specific set of external inputs U_α . The left side of the figure presents the framework of a metaheuristic algorithm that applies chaotic transcription. Unlike fixed-length metaheuristic methods, this approach requires transforming individual information from the DNA population into the RNA population before evaluation. The right side of the Figure details the chaotic transcription process. The three upper matrices represent DNA individuals D_1 , D_2 , and D_3 , each corresponding to a neuron. The first row records threshold values p , while the second row stores weight information z . The PWLCM is then used to decompress this information. The decompression process follows these steps. First, the integer-rounded value of m_i in a DNA individual determines the number of weights and thresholds in a neuron. For example, D_1 has five, D_2 has three, and D_3 has two. Next, the second dimension value $z_{i,1}$ and the third dimension value y_i from the first row serve as the initial value and parameter for PWLCM, generating the required thresholds. The same method applies to the second row to compute the weights. These weights and thresholds define the activation state of each dendrite in the neuron. By introducing chaotic transcription, the neural structure achieves variable dimensionality, significantly improving population diversity.

Figure 7 takes a wind farm with 9 possible placement positions as an example, showing the specific execution process of chaotic transcription in WFLOP. In the 3×3 grid, the blue sections represent the positions for placing turbines, while the green positions are vacant. The left side of the figure, similar to Fig. 6, represents the operational framework of the metaheuristic algorithm. The right side illustrates the chaotic transcription process, which extracts wind farm layout information from DNA population individuals. The three upper vectors, with identical

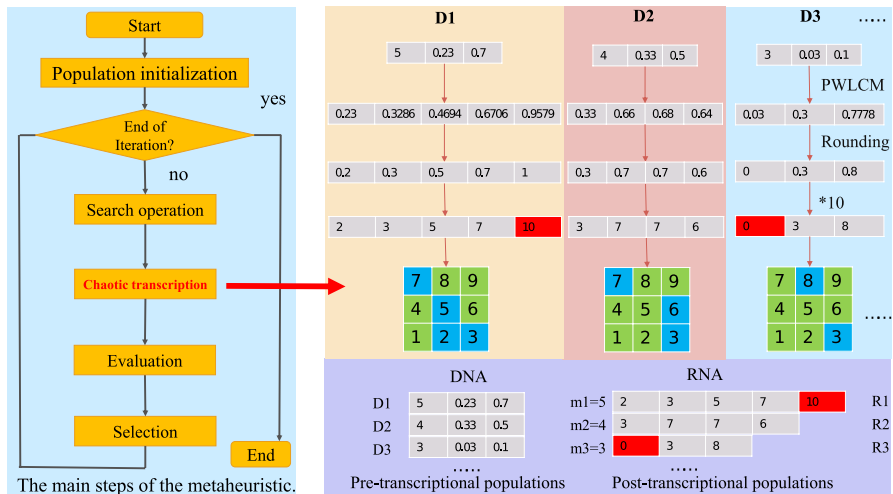


Fig. 7 The execution step of the chaotic transcription in WFLOP

dimensions, correspond to DNA individuals D_1 , D_2 , and D_3 , each representing a different wind farm layout. The PWLCM is then used to decompress this information. The decompression process follows these steps. First, the integer-rounded value of the first dimension m_i in a DNA individual determines the number of wind turbines in the layout. For example, layout D_1 contains five turbines, D_2 contains three, and D_3 contains two. Once the number of turbines is set, the second dimension value $z_{i,1}$ and the third dimension value y_i from the first row of the DNA individual are input into PWLCM as the initial value and parameter, respectively, to compute the position of each turbine. These computed positions are then converted into actual locations in the wind farm layout through rounding operations, enabling layout optimization.

5 Experimental results

To thoroughly validate the feasibility and practical performance of chaotic map encoding in metaheuristics, we conducted experiments on WFLOP [77] and DNM optimization problems [71]. The key aspects of the experiments were as follows:

1. To ascertain whether metaheuristic algorithms encoded with chaotic maps exhibit superiority over traditional algorithms when confronting fixed-length problems.
2. To verify if metaheuristic algorithms encoded with chaotic maps can effectively handle metameric variable-length problems.
3. To assess whether metaheuristic algorithms encoded with chaotic maps outperform mainstream variable-length genetic algorithms.

The experimental results comprehensively confirm the feasibility and effectiveness of chaotic map encoding. All experiments are performed using MATLAB on a PC with a 3.00 GHz Intel(R) Core(TM) i7-9700 CPU and 36 GB of RAM.

5.1 Performance evaluation criteria

To evaluate the performance of the algorithms, we employed the following assessment tools:

1. *The Wilcoxon Rank-sum Test* (significance level $\alpha = 0.05$): This non-parametric test is used to determine if two independent samples likely come from the same distribution. It contrasts this possibility against an alternative hypothesis. We apply this test to discern differences between chaotic map-based metaheuristics and their traditional counterparts. After 30 optimization trials, the outcomes are presented using the notation $W/T/L$, where: W denotes instances where chaotic map-based metaheuristics significantly outperform traditional ones; L identifies cases where traditional metaheuristics surpass the chaotic map-based ones; T indicates scenarios with negligible performance differences between the two approaches. In tabular representations, an algorithm that outperforms others is marked with “+”, comparable performance with “=”, and underperformance with “-”.
2. *Friedman Test* (significance level $\alpha = 0.05$): This is a non-parametric statistical method employed to evaluate the performance differences between algorithms by comparing their medians. A higher Score value in this study is indicative of superior algorithm performance.
3. *Wilcoxon signed-rank test* (significance level $\alpha = 0.05$): The Wilcoxon signed-rank test is a non-parametric statistical test used to compare two paired or related samples. It evaluates whether the median difference between pairs of observations is zero, without assuming the normality of the data distribution.
4. *Convergence curve*: This graphical tool tracks the progression of optimal values through iterative processes. The x -axis represents iteration counts, while the y -axis highlights either energy yield or turbine count, providing a visual representation of algorithmic efficiency over time.
5. *Box-and-whisker plots*: These plots offer a comprehensive view of data distribution. The top and bottom lines of the box signify the highest and lowest values, respectively. The box itself encompasses the first and third quartiles, while the median value is denoted by a distinct red line. Outliers are marked with a red “+” symbol. A substantial disparity between the maximum and minimum values typically indicates unstable in the algorithm’s performance.

5.2 Dendritic neuron model optimization

5.2.1 Dataset description and parameter settings

In this study, we utilized ten commonly employed classification datasets from the UCI machine learning repository [78]. These datasets encompass Tic-tac-toe, Heart, Australia, Congress, Vote, Spect, German, Breast, Ionosphere, and KrVsKpEW. Additionally, the DNM model requires three hyper-parameters to be predetermined: the number of dendrites m , the distance factor k , and the firing parameter q . To efficiently determine the optimal combination of these parameters, the Taguchi orthogonal experimental design method [71, 79] is applied. Unlike full factorial analysis, which examines all possible combinations of factors and levels, the Taguchi approach significantly reduces the number of experiments by focusing on a subset of orthogonal combinations, achieving a balance between efficiency and statistical robustness. In DNM, each hyper-parameter is assigned five levels: $m \in \{1, 3, 5, 10, 20\}$, $k \in \{1, 5, 10, 15, 20\}$, and $q \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. Performing a full-factorial analysis would require $5^3 = 125$ experiments to exhaustively identify the best combination, which is computationally expensive. Alternatively, the Taguchi method employs an orthogonal array $L_{25}(5^3)$ containing 25 carefully chosen combinations, as shown in Table 1. This method effectively determines the statistically optimal hyper-parameter settings with significantly fewer trials.

These 25 parameter combinations are not the specific settings for this study but represent the parameter configurations required to tune fixed-length metaheuristic algorithms for optimizing DNM. They serve as a control group for comparison with our proposed method. Specifically, our approach leverages a chaos transcription method that enables variable-length metaheuristic algorithms to handle the optimization problem in a single execution, matching the performance of traditional

Table 1 Orthogonal experimental design ($L_{25}(5^3)$) method for hyper-parameters in DNM

No.	m	k	q	No.	m	k	q
#1	1	1	0.1	#13	5	10	0.9
#2	1	5	0.3	#14	5	15	0.1
#3	1	10	0.5	#15	5	20	0.3
#4	1	15	0.7	#16	10	1	0.7
#5	1	20	0.9	#17	10	5	0.9
#6	3	1	0.3	#18	10	10	0.1
#7	3	5	0.5	#19	10	15	0.3
#8	3	10	0.7	#20	10	20	0.5
#9	3	15	0.9	#21	20	1	0.9
#10	3	20	0.1	#22	20	5	0.1
#11	5	1	0.5	#23	20	10	0.3
#12	5	5	0.7	#24	20	15	0.5
				#25	20	20	0.7

fixed-length metaheuristics that rely on the Taguchi orthogonal experimental design method. Notably, the parameter combinations for full-factorial analysis consist of 125 settings. In other words, our research allows the algorithm to achieve comparable solution quality while requiring only one run instead of 125, significantly reducing computational resources. Through the chaos transcription technique, we successfully encoded both the hyper-parameters m , k , q and internal parameters z , p of DNM, enabling the model to evolve without any manual tuning throughout the optimization process.

5.2.2 Comparison between chaotic map encoded metaheuristics and fixed-length metaheuristics

The experiments in this section aim to demonstrate that chaotic map-encoded metaheuristics can effectively save computational resources consumed in experiments by automatically adjusting the dimensions of the problem. In the experiment, the number of evaluation times is 3×10^6 , and the number of the population is 100. Each algorithm is executed independently 30 times on an identical computer setup. The parameters for both chaotic map-encoded metaheuristics and fixed-length metaheuristics are set according to their respective original papers, ensuring absolute fairness in the experiments. The dataset is split with 70% used for training and the remaining 30% designated for testing. In this section, we introduce the chaotic map enhanced GA (CGA) and SHADE (CSHADE). As shown in Tables 2 and 3, larger classification accuracy values and higher ranks indicate better overall performance across the evaluated datasets. Here, W/T/L represents the outcomes of the Wilcoxon Rank-Sum Test [80], recording the number of wins (+), ties (=), and losses (−) for each comparison. Rank denotes the algorithm ranking obtained from the Friedman test. Let p_o and p_c denote the p -values before and after Holm–Bonferroni correction [81, 82], respectively. The bold values in the tables indicate the results obtained by the proposed method in this study, which outperform those of the traditional algorithms used for comparison. Any adjusted p -values less than the significance level α are considered significant, meaning the corresponding null hypotheses are rejected. The Holm–Bonferroni procedure starts by ordering the original p -values p_o in ascending order as $p_o(1), p_o(2), \dots, p_o(m)$, where m is the total number of hypotheses tested. The smallest adjusted p -value is computed as

$$p_c(1) = p_o(1) \cdot m. \quad (14)$$

For $i = 2$ to m , each subsequent adjusted p -value $p_c(i)$ is defined recursively as:

$$p_c(i) = \max(p_o(i) \cdot (m - i + 1), p_c(i - 1)). \quad (15)$$

This recursive definition ensures that the adjusted p -values are non-decreasing with respect to the ordering of p_o . It is important to note that p_c is no longer a probability of observing specific or more extreme data under the assumption that the null hypothesis is true. The p_c does not have interpretation of probability in any sense (e.g., it can be greater than 1). Its interpretation requires the context of a specific

Table 2 The classification accuracy between DNM trained using CGA and GA

CGA	#1		#2		#3		#4		#5	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
Australia	0.805	0.024	0.444	0.000	+	0.652	0.128	+	0.698	0.103
BreastEW	0.693	0.092	0.635	0.000	+	0.635	0.000	+	0.635	0.000
CongressEW	0.861	0.024	0.638	0.000	+	0.671	0.075	+	0.762	0.109
German	0.713	0.032	0.267	0.000	+	0.267	0.000	+	0.267	0.001
Heart	0.784	0.019	0.539	0.000	+	0.822	0.018	-	0.822	0.031
Ionosphere	0.699	0.112	0.179	0.000	+	0.179	0.000	+	0.179	0.000
KrVsKpEW	0.558	0.072	0.473	0.000	+	0.473	0.000	+	0.473	0.000
SpectEW	0.796	0.042	0.920	0.000	-	0.920	0.000	-	0.920	0.000
Tic-tac-toe	0.642	0.034	0.393	0.072	+	0.660	0.005	-	0.664	0.013
Vote	0.804	0.035	0.608	0.000	+	0.610	0.005	+	0.753	0.103
W/TL	-/-		9/0/1			8/0/2			7/0/3	
Rank	18		8.6			11.05			14.75	
P_o	NaN		0.010			0.027			0.105	
P_c	NaN		0.244			0.438			1.160	
CGA	#6		#7		#8		#9		#10	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
Australia	0.444	0.000	+	0.444	0.000	+	0.562	0.118	+	0.845
BreastEW	0.635	0.000	+	0.635	0.000	+	0.635	0.000	+	0.769
CongressEW	0.638	0.000	+	0.638	0.000	+	0.647	0.030	+	0.951
German	0.267	0.000	+	0.267	0.000	+	0.267	0.000	+	0.640
Heart	0.539	0.000	+	0.575	0.050	+	0.750	0.045	+	0.847
Ionosphere	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+	0.230
KrVsKpEW	0.473	0.000	+	0.473	0.000	+	0.473	0.000	+	0.479
CGA	#11		#12		#13		#14		#15	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
Australia	0.655	0.114	+	0.655	0.114	+	0.655	0.114	+	0.655
BreastEW	0.635	0.000	+	0.635	0.000	+	0.635	0.000	+	0.635
CongressEW	0.715	0.100	+	0.715	0.100	+	0.715	0.100	+	0.715
German	0.267	0.000	+	0.267	0.000	+	0.267	0.000	+	0.267
Heart	0.788	0.045	=	0.788	0.045	=	0.788	0.045	=	0.788
Ionosphere	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+	0.179
KrVsKpEW	0.473	0.000	+	0.473	0.000	+	0.473	0.000	+	0.473
SpectEW	0.920	0.000	-	0.920	0.000	-	0.920	0.000	-	0.920
Tic-tac-toe	0.670	0.013	-	0.670	0.013	-	0.670	0.013	-	0.670
Vote	0.663	0.094	+	0.663	0.094	+	0.663	0.094	+	0.663
W/TL	7/1/2			7/1/2			7/1/2			7/1/2
Rank	13.9			13.9			13.9			13.9
P_o	0.049			0.049			0.049			0.049
P_c	0.635			0.635			0.635			0.635

Table 2 (continued)

	#6		#7		#8		#9		#10		#11				
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std			
SpectEW	0.920	0.000	-	0.920	0.000	-	0.920	0.000	-	0.754	0.039	+	0.920	0.000	-
Tic-tac-toe	0.342	0.000	+	0.658	0.010	-	0.671	0.011	-	0.667	0.021	-	0.342	0.000	+
Vote	0.608	0.000	+	0.608	0.000	+	0.616	0.026	+	0.613	0.017	+	0.914	0.026	+
W/T/L	9/0/1		8/0/2		8/0/2		8/0/2		8/0/2		4/0/6		9/0/1		+
Rank	8.35		9.5		12.9		13.2		13.2		22.3		8.35		
p_o	0.010		0.020		0.027		0.037		0.037		0.846		0.010		
p_c	0.244		0.371		0.438		0.520		0.520		3.848		0.234		
#12	#13		#14		#15		#16		#17						
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std			
Australia	0.444	0.000	+	0.444	0.000	+	0.844	0.012	-	0.844	0.031	-	0.444	0.000	+
BreastEW	0.635	0.000	+	0.635	0.000	+	0.660	0.057	=	0.665	0.070	=	0.635	0.000	+
CongressEW	0.638	0.000	+	0.638	0.000	+	0.946	0.021	-	0.943	0.024	-	0.638	0.000	+
German	0.267	0.000	+	0.267	0.000	+	0.547	0.080	+	0.460	0.140	+	0.267	0.000	+
Heart	0.543	0.011	+	0.601	0.065	+	0.846	0.009	-	0.840	0.014	-	0.539	0.001	+
Ionosphere	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+
KrVsKpEW	0.473	0.000	+	0.473	0.000	+	0.478	0.031	+	0.472	0.001	+	0.473	0.000	+
SpectEW	0.920	0.000	-	0.920	0.000	-	0.741	0.042	+	0.754	0.064	+	0.920	0.000	-
Tic-tac-toe	0.648	0.035	=	0.664	0.011	-	0.679	0.034	-	0.664	0.017	-	0.342	0.000	+
Vote	0.608	0.000	+	0.609	0.002	+	0.909	0.032	-	0.894	0.041	-	0.608	0.000	+
W/T/L	8/1/1		8/0/2		4/1/5		4/1/5		4/1/5		9/0/1		9/0/1		+
Rank	9.3		10.55		20.75		17.15		17.15		8.35		9		
p_o	0.020		0.020		0.846		0.695		0.695		0.010		0.014		
p_c	0.371		0.352		3.383		5.625		5.625		0.225		0.273		
Australia	0.841	0.018	-	0.836	0.033	-	0.814	0.053	-	0.444	0.000	+	0.785	0.057	-
													0.826	0.042	-

Table 2 (continued)

	#18		#19		#20		#21		#22		#23	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
BreastEW	0.635	0.000	+	0.640	0.025	+	0.635	0.000	+	0.635	0.000	+
CongressEW	0.926	0.030	–	0.940	0.034	–	0.638	0.000	+	0.829	0.082	–
German	0.317	0.060	+	0.347	0.089	+	0.267	0.000	+	0.267	0.001	+
Heart	0.843	0.008	–	0.845	0.010	–	0.539	0.000	+	0.813	0.021	–
Ionosphere	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+	0.179	0.000	+
KrVsKpEW	0.473	0.000	+	0.473	0.000	+	0.473	0.000	+	0.473	0.000	+
SpectEW	0.861	0.065	–	0.805	0.126	–	0.920	0.000	–	0.920	0.001	–
Tic-tac-toe	0.702	0.026	–	0.669	0.022	–	0.342	0.000	+	0.670	0.017	–
Vote	0.888	0.036	–	0.900	0.037	–	0.608	0.000	+	0.754	0.085	+
W/TT/L	4/0/6			4/1/5			9/0/1		5/2/3		4/0/6	
Rank	17.6			18.4			8.35		14.55		15.9	
P_o	0.922			0.846			0.010		0.160		0.770	
P_c	1.691			2.537			0.215		1.601		5.387	
	#24		#25									
	Mean	Std	Mean	Std								
Australia	0.801	0.076	–	0.792	0.076	–						
BreastEW	0.635	0.000	+	0.635	0.000	+						
CongressEW	0.930	0.037	–	0.903	0.075	–						
German	0.298	0.067	+	0.280	0.026	+						
Heart	0.835	0.020	–	0.817	0.022	–						
Ionosphere	0.179	0.000	+	0.179	0.000	+						
KrVsKpEW	0.473	0.000	+	0.473	0.000	+						
SpectEW	0.908	0.026	–	0.914	0.014	–						

Table 2 (continued)

	#24		#25	
	Mean	Std	Mean	Std
Tic-tac-toe	0.673	0.020	–	0.658 0.023 –
Vote	0.871	0.047	–	0.846 0.080 –
W/T/L	4/0/6			4/1/5
Rank	16.5			14.3
P_o	0.770			0.625
P_c	4.617			5.625

The bold values indicate the results obtained by the proposed method in this study, which outperform those of the compared traditional algorithms

Table 3 The classification accuracy between DNM trained using CSHADE and SHADE

No.	W/T/L	Rank	p_o	p_c	No.	W/T/L	Rank	p_o	p_c
CSHADE	—/—/—	11.9	NaN	NaN	#13	3/0/7	15.45	0.006	0.135
#1	8/1/1	6.15	0.010	0.215	#14	2/2/6	18.05	0.049	0.830
#2	1/1/8	18	0.049	0.830	#15	3/3/4	11.25	0.492	5.906
#3	2/0/8	17.2	0.432	6.043	#16	8/1/1	6.25	0.084	1.260
#4	3/3/4	10.45	0.846	5.074	#17	4/0/6	18.8	0.625	5.625
#5	3/4/3	9.35	0.846	5.074	#18	0/2/8	19.9	0.492	5.906
#6	8/1/1	5.95	1.000	3.383	#19	3/3/4	12.8	0.010	0.205
#7	1/1/8	20.8	0.010	0.215	#20	5/1/4	11.15	0.695	5.000
#8	2/1/7	15.55	0.004	0.094	#21	8/0/2	7.25	0.002	0.049
#9	4/0/6	11.1	0.432	6.043	#22	0/1/9	23.55	0.014	0.260
#10	3/2/5	14.35	1.000	3.000	#23	0/2/8	17.5	0.846	4.229
#11	8/1/1	5.65	1.000	2.000	#24	4/2/4	12.2	0.492	5.414
#12	1/1/8	20.8	0.625	5.625	#25	4/2/4	9.6	0.014	0.260

The bold values indicate the results obtained by the proposed method in this study, which outperform those of the compared traditional algorithms

family of tests and alpha. The volume of the difference between the corrected p -value and alpha does not matter/does not have meaning. It only matters whether one is greater than the other. The Friedman test results indicate that the p -value for CGA is 2.63×10^{-10} and for CSHADE is 2.48×10^{-14} , demonstrating a statistically significant difference among the algorithms (significance level $\alpha = 0.05$). Furthermore, the Friedman ranking results show that the CGA-trained DNM ranks 4th among 26 methods, while the CSHADE-trained DNM ranks 15th.

Further analysis of the pairwise comparisons reveals that, before Holm–Bonferroni correction, the p -values p_o , obtained from the Wilcoxon signed-rank test [83], indicate significant differences between our method and traditional algorithms. However, after Holm–Bonferroni correction (p_c), only one algorithm remains statistically significant compared to ours. It is important to note that these 25 fixed-length metaheuristic algorithms correspond to the optimization results of DNM under different parameter settings, meaning that each fixed-length algorithm undergoes 25 rounds of parameter tuning to obtain its best solutions. Therefore, this study essentially compares our proposed variable-length metaheuristic algorithm under equal computational resources (1×) against traditional algorithms running under 25× computational resources. While our method may not show significant superiority in terms of the mean-based evaluation, its computational cost is only 1/25 of traditional methods, highlighting a remarkable improvement in computational efficiency.

To further validate the practical advantages of the proposed algorithm, we employ the Wilcoxon Rank-sum Test for pairwise comparisons. Unlike the Friedman test,

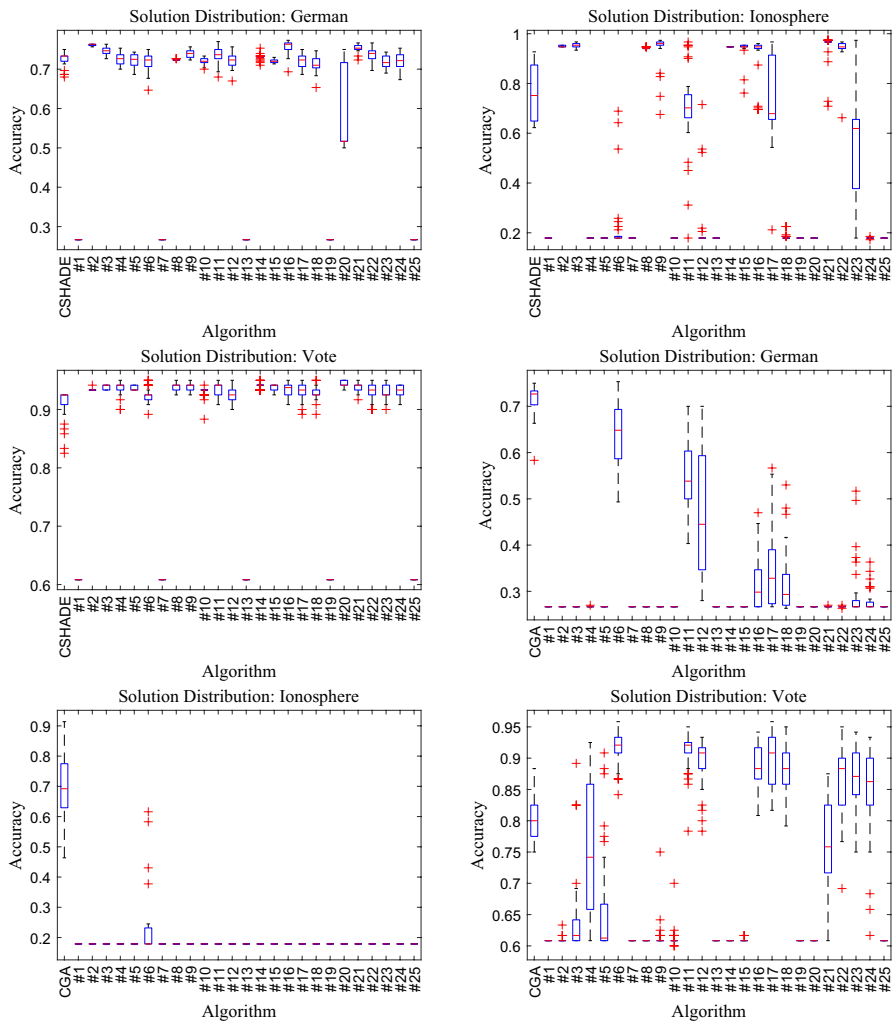


Fig. 8 The Box-and-whisker diagrams of the accuracy

which assesses the overall ranking of multiple algorithms, the Wilcoxon Rank-sum Test is more suitable for analyzing result distributions across multiple test problems, particularly in the presence of non-normally distributed data. This method allows us to identify specific cases where our algorithm exhibits statistical advantages, even if the overall mean differences are not significant. Specifically, if our method demonstrates superior performance on some of the 26 test problems, but is affected by a few outliers, the mean may not fully reflect the algorithm's strengths. In such cases, the Wilcoxon test effectively captures the overall distribution trend, revealing

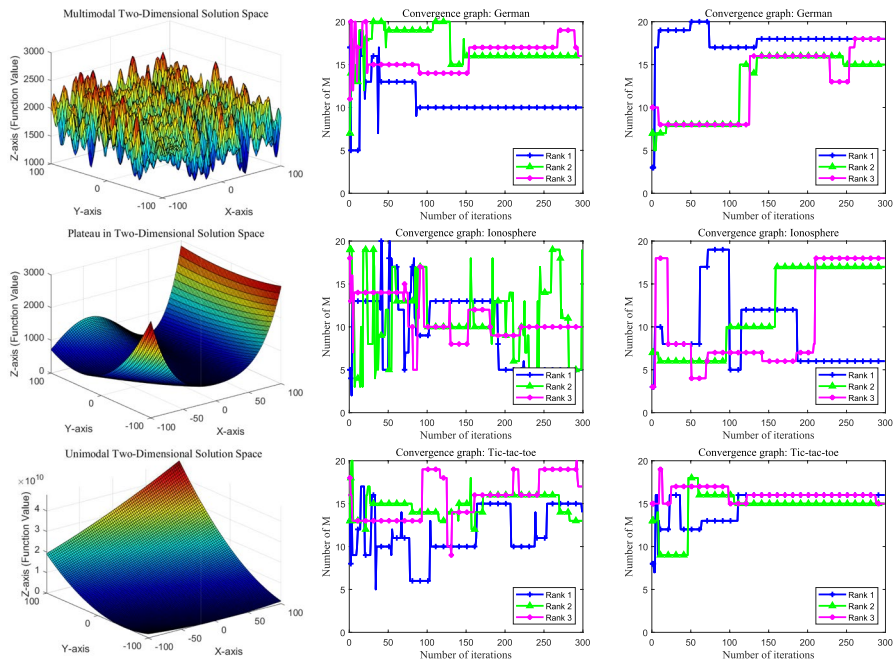


Fig. 9 The progression of m values throughout iterative stages. The second column presents the convergence graph of m for CSHADE, while the third column displays that of CGA. The first column presents the inferred potential solution space configuration based on the characteristics of the convergence graph

the statistical advantages of the new algorithm even when the mean differences are not significant. Experimental results confirm that our approach improves the performance of DNM on multiple datasets.

In summary, our chaotic transcription-based metaheuristic algorithm demonstrates competitive performance in both Friedman and Wilcoxon tests, despite consuming only 1/25 of the computational resources required by traditional methods. Although the pairwise comparisons after Holm–Bonferroni correction fail to reach statistical significance, considering the substantial 25-fold reduction in computational cost, we argue that this study provides significant practical value. Of course, there is still room for further performance improvements. Nevertheless, under the constraint of significantly lower computational resource consumption, our method has already exhibited strong application potential, offering an efficient new approach for large-scale optimization problems.

In addition, while CSHADE conserves computational resources and offers some improvements, its performance enhancement is less pronounced compared to CGA.

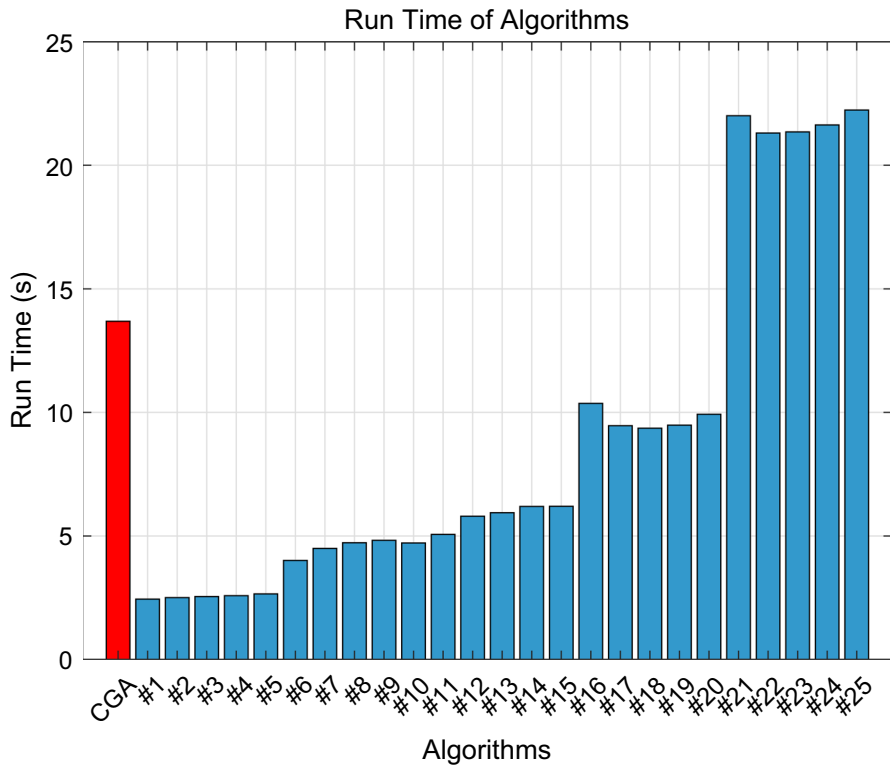


Fig. 10 The average runtime of CGA and the traditional GA with 25 fixed-length parameters

This limitation is likely due to the inherent accuracy constraints of DNM, as suggested by Fig. 8. This figure illustrates the boxplot of DNM accuracies trained with CSHADE, CGA, SHADE, and GA across German, Ionosphere, and Vote datasets. It is clear that CGA notably outperforms GA in most settings. While CSHADE does not exhibit a significant advantage over traditional SHADE in most parameter combinations, it also does not demonstrate any significant disadvantages.

Figure 9 presents a comprehensive analysis of the iterative behavior of the parameter m across various datasets, illustrating distinct convergence trends in different problem contexts. For the German dataset, there is a notable variation in the m values among the top three ranked DNMs in both algorithms, indicating a solution space characterized by multiple local optima. In the Ionosphere dataset, the m values for the top-ranked DNMs not only differ significantly but also show considerable fluctuations during the algorithms' iterative processes, suggesting a solution space with potential plateaus. In contrast, the Tic-tac-toe dataset demonstrates relatively minor variations in the m values among the top DNMs, implying an unimodal solution space.

Figure 10 presents the average runtime of CGA and the traditional GA with 25 fixed-length parameter setups over 30 runs on the Vote dataset. The average runtime of CGA is 13.69 s, while the total runtime of the traditional GA is 221.79 s, which is 16.2 times longer than CGA. This differs from our previously mentioned 1/25 ratio. The reason is that for the DNM optimization problem, an increased number of dendrites m leads to a more complex neuronal structure, which improves classification accuracy. As a result, CGA tends to favor larger m values. Since m is the main factor affecting DNM runtime, Fig. 9 also shows that a higher m results in longer execution times. However, for other problems, a lower number of homogeneous components may be preferable, leading to computation time savings greater than 25 times.

Algorithm 2 Variable Length Genetic Algorithm (GA)

Require: Initialized parameters and data structures
Ensure: Optimized solution

```

1: /* Initialization operation */
2: for  $i = 1$  to  $n$  do
3:   Randomly determine the dimension (number of turbines) of the individual
4:   Initialize the features of the individual
5:   Evaluate the fitness value of each individual using  $f(X_i)$ 
6: end for
7: while Terminal condition is not met do
8:   Select elite individuals
9:   /* Perform crossover operation */
10:  Determine crossover point (considering the minimum dimension of parents)
11:  Generate crossover individuals (dimensions may differ from parents)
12:  /* Perform mutation operation */
13:  Randomly determine the dimension of the new individual
14:  Generate mutated individuals
15:  Combine elite individuals, crossover individuals, and mutated individuals
16:  Boundary detection
17:  for  $i = 1$  to  $n$  do
18:    Evaluate the fitness value of each individual using  $f(X_i)$ 
19:  end for
20:  /* Selection operation */
21:  for  $i = 1$  to  $n$  do
22:    Execute Eq. (8)
23:  end for
24: end while

```

5.2.3 Comparison with variable length genetic algorithm

The experiments in this section have demonstrated that metaheuristics encoded with chaotic maps outperform variable-length genetic algorithms (VGA) in DNM optimization problems. In VGA, the variation in individual dimensions is achieved by randomly initializing individuals of different dimensions during the mutation process,

Table 4 The classification accuracy between DNM trained using CSHADE and VGA

	Mean			Best		Worst	
	CSHADE	VGA		CSHADE	VGA	CSHADE	VGA
Australia	0.842	0.828	=	0.860	0.860	0.826	0.749
BreastEW	0.894	0.638	+	0.953	0.688	0.841	0.635
CongressEW	0.949	0.898	+	0.969	0.962	0.938	0.808
German	0.725	0.379	+	0.753	0.550	0.683	0.267
Heart	0.829	0.834	−	0.852	0.872	0.815	0.788
Ionosphere	0.769	0.225	+	0.954	0.589	0.623	0.179
KrVsKpEW	0.735	0.477	+	0.818	0.530	0.642	0.473
SpectEW	0.791	0.832	−	0.856	0.920	0.717	0.599
Tic-tac-toe	0.665	0.737	−	0.710	0.791	0.614	0.655
Vote	0.910	0.874	+	0.933	0.950	0.867	0.783
W/T/L	6/1/3			−/−/−		−/−/−	
p_o	0.105			0.359		0.006	

The bold values indicate the results obtained by the proposed method in this study, which outperform those of the compared traditional algorithms

and by selecting random loci for exchanging gene segments during the crossover process. The pseudocode of this algorithm is presented in Algorithm 2 [16]. In this section, we apply CSHADE and VGA to the variable-length DNM optimization problem, with the comparative results of both algorithms presented in Table 4. In this table, we evaluated the performance of the two algorithms on 10 datasets, each executed over 30 independent runs. In addition to comparing the mean classification accuracy, we also analyzed the best and worst accuracy values obtained across the runs. The results indicate that, although our method does not show statistically significant improvements in mean or best-case accuracy compared to the traditional variable-length VGA, it achieves a statistically significant enhancement in worst-case performance ($p_o < 0.05$). This suggests that our method offers better robustness and stability. We believe this improvement in the lower bound may be attributed to the ergodicity of chaotic maps, which helps the algorithm avoid becoming trapped in local optima. We plan to explore this phenomenon further in future work to better adapt chaotic mechanisms for performance enhancement.

5.3 Wind farm layout optimization problem

In experiments, GLPSO, SHADE, and SIS are selected, along with their improved versions based on chaotic maps (CGLPSO, CSHADE, and CSIS), for comparison. Each algorithm's population consists of 100 individuals, with 2×10^6 evaluation times and 30 runs. All algorithms are evaluated across 13 wind farms with various numbers of wind turbines under four different wind distributions: P_1 , P_2 , P_3 , and P_4 . Figure 4 illustrates 12 wind farm layouts, considering landowner constraints. Light green cells represent potential turbine locations, while shaded cells are non-viable

Table 5 Comparison of η between CGLPSO, CSHADE, CSIS and GLPSO, SHADE, SIS under wind profile P_1 , P_2 , P_3 , and P_4

	P_1			P_2			P_3			P_4		
	W/T/L	P_o	P_c	W/T/L	P_o	P_c	W/T/L	P_o	P_c	W/T/L	P_o	P_c
CGLPSO vs. GLPSO												
$m_i=15$	13/0/0	0.0002	0.0029	0/9/4	0.0942	0.6597	13/0/0	0.0002	0.0029	0/8/5	0.0134	0.0940
$m_i=20$	13/0/0	0.0002	0.0029	2/9/2	0.4973	1.6572	13/0/0	0.0002	0.0029	0/8/5	0.0327	0.1331
$m_i=25$	13/0/0	0.0002	0.0027	3/9/1	0.3396	2.0376	13/0/0	0.0002	0.0027	0/9/4	0.0034	0.0376
$m_i=30$	13/0/0	0.0002	0.0024	0/12/1	0.4143	1.6980	13/0/0	0.0002	0.0024	0/10/3	0.0024	0.0293
$m_i=35$	13/0/0	0.0002	0.0022	1/10/2	0.9460	1.6787	13/0/0	0.0002	0.0022	3/8/2	0.3396	0.3396
$m_i=40$	12/1/0	0.0002	0.0020	0/11/2	0.8394	1.6787	13/0/0	0.0002	0.0020	6/6/1	0.0398	0.1309
$m_i=45$	12/1/0	0.0002	0.0017	0/12/1	0.3396	2.0376	13/0/0	0.0002	0.0017	8/3/2	0.0398	0.1194
$m_i=50$	12/1/0	0.0005	0.0020	0/10/3	0.0046	0.0371	12/1/0	0.0005	0.0024	9/3/1	0.0266	0.1331
$m_i=55$	12/1/0	0.0002	0.0015	0/8/5	0.0017	0.0154	12/1/0	0.0005	0.0024	9/3/1	0.0134	0.0940
$m_i=60$	11/2/0	0.0005	0.0020	0/4/9	0.0012	0.0122	12/0/1	0.0012	0.0037	11/1/1	0.0061	0.0488
$m_i=65$	11/2/0	0.0005	0.0015	0/5/8	0.0005	0.0054	12/0/1	0.0034	0.0068	12/0/1	0.0046	0.0417
$m_i=70$	9/4/0	0.0017	0.0017	0/2/11	0.0002	0.0029	8/4/1	0.0215	0.0215	12/0/1	0.0034	0.0376
Total	367/178/79											
CSHADE vs. SHADE												
$m_i=15$	12/1/0	0.0002	0.0029	0/0/13	0.0002	0.0029	13/0/0	0.0002	0.0029	0/0/13	0.0002	0.0029
$m_i=20$	13/0/0	0.0002	0.0029	0/0/13	0.0002	0.0029	13/0/0	0.0002	0.0029	0/0/13	0.0002	0.0029
$m_i=25$	13/0/0	0.0002	0.0027	0/0/13	0.0002	0.0027	13/0/0	0.0002	0.0027	0/0/13	0.0002	0.0027
$m_i=30$	13/0/0	0.0002	0.0024	0/0/13	0.0002	0.0024	13/0/0	0.0002	0.0024	0/0/13	0.0002	0.0024
$m_i=35$	13/0/0	0.0002	0.0022	0/0/13	0.0002	0.0022	13/0/0	0.0002	0.0022	0/0/13	0.0002	0.0022
$m_i=40$	13/0/0	0.0002	0.0020	0/0/13	0.0002	0.0020	13/0/0	0.0002	0.0020	0/0/13	0.0002	0.0020
$m_i=45$	13/0/0	0.0002	0.0017	0/0/13	0.0002	0.0017	13/0/0	0.0002	0.0017	1/0/12	0.0005	0.0029
$m_i=50$	13/0/0	0.0002	0.0015	0/0/13	0.0002	0.0015	13/0/0	0.0002	0.0015	1/2/10	0.0017	0.0085
$m_i=55$	13/0/0	0.0002	0.0012	0/0/13	0.0002	0.0012	13/0/0	0.0002	0.0012	13/0/0	0.0002	0.0012

Table 5 (continued)

	P_1			P_2			P_3			P_4		
	W/T/L	p_o	p_c	W/T/L	p_o	p_c	W/T/L	p_o	p_c	W/T/L	p_o	p_c
$m_i=60$	13/0/0	0.0002	0.0010	11/2/0	0.0005	0.0010	13/0/0	0.0002	0.0010	13/0/0	0.0002	0.0010
$m_i=65$	13/0/0	0.0002	0.0007	11/2/0	0.0002	0.0010	13/0/0	0.0002	0.0007	13/0/0	0.0002	0.0007
$m_i=70$	13/0/0	0.0002	0.0005	9/4/0	0.0005	0.0010	13/0/0	0.0002	0.0005	13/0/0	0.0002	0.0005
Total	309/20/295											
<i>CSIS vs. SIS</i>												
$m_i=15$	13/0/0	0.0002	0.0029	13/0/0	0.0002	0.0029	13/0/0	0.0002	0.0029	12/1/0	0.0002	0.0029
$m_i=20$	13/0/0	0.0002	0.0029	13/0/0	0.0002	0.0029	13/0/0	0.0002	0.0029	12/1/0	0.0002	0.0029
$m_i=25$	13/0/0	0.0002	0.0027	13/0/0	0.0002	0.0027	13/0/0	0.0002	0.0027	12/1/0	0.0002	0.0027
$m_i=30$	13/0/0	0.0002	0.0024	13/0/0	0.0002	0.0024	13/0/0	0.0002	0.0024	11/2/0	0.0002	0.0024
$m_i=35$	13/0/0	0.0002	0.0022	13/0/0	0.0002	0.0022	13/0/0	0.0002	0.0022	13/0/0	0.0002	0.0022
$m_i=40$	13/0/0	0.0002	0.0020	13/0/0	0.0002	0.0020	13/0/0	0.0002	0.0020	12/1/0	0.0002	0.0020
$m_i=45$	13/0/0	0.0002	0.0017	12/1/0	0.0002	0.0017	13/0/0	0.0002	0.0017	13/0/0	0.0002	0.0017
$m_i=50$	13/0/0	0.0002	0.0015	12/1/0	0.0002	0.0015	13/0/0	0.0002	0.0015	13/0/0	0.0002	0.0015
$m_i=55$	13/0/0	0.0002	0.0012	13/0/0	0.0002	0.0012	13/0/0	0.0002	0.0012	13/0/0	0.0002	0.0012
$m_i=60$	13/0/0	0.0002	0.0010	11/2/0	0.0005	0.0010	13/0/0	0.0002	0.0010	13/0/0	0.0002	0.0010
$m_i=65$	13/0/0	0.0002	0.0007	11/2/0	0.0002	0.0010	13/0/0	0.0002	0.0007	13/0/0	0.0002	0.0007
$m_i=70$	13/0/0	0.0002	0.0005	9/4/0	0.0005	0.0010	13/0/0	0.0002	0.0005	13/0/0	0.0002	0.0005
Total	608/16/0											

The bold values indicate the results obtained by the proposed method in this study, which outperform those of the compared traditional algorithms

for turbine installation. Group 1 wind farms $L1$ – $L6$ offer 120 available cells, Group 2 wind farms $L7$ – $L12$ have 132 cells, and the baseline scenario $L0$ provides 144 cells. Each farm spans an area of $3,415,104 \text{ m}^2$, with cell widths of 154 m .

5.3.1 Wind farm layout optimization with fixed turbine count

The optimization results in this section demonstrate that metaheuristic algorithms encoded with chaotic maps retain exceptional performance when dealing with fixed-length problems. Table 5 presents the performance comparison between chaotic map-coded algorithms and traditional algorithms for four wind conditions, P_1 to P_4 , across a range of turbine numbers, $m_i = [15, 20, \dots, 70]$. The $W/T/L$ values in the table indicate a clear superiority of the CGLPSO, CSHADE, and CSIS over their original counterparts. Specifically, CGLPSO outperforms GLPSO in 33 of 48 cases, CSHADE excels over SHADE in 26 cases, and CSIS surpasses SIS in all instances. Furthermore, each scenario is subdivided into 13 different wind farm types, denoted as $[L0, L1, \dots, L12]$, leading to a total of 624 comparative conditions per algorithm pair. Cumulative results show CGLPSO holds a significant advantage in 367 conditions, CSHADE in 309, and CSIS in 608 conditions, underscoring the substantial improvement brought by chaotic map to metaheuristic performance. In addition, the corrected p -values p_c obtained through the Holm–Bonferroni correction indicate that CGLPSO and CSIS significantly outperform GLPSO and SIS in more than half of the wind farm scenarios, while CSHADE falls slightly below this threshold. These results suggest that the performance of the chaotic transcription technique still requires further improvement, particularly through adaptation to the specific characteristics of each algorithm.

To elucidate Table 5, we focus on two specific scenarios: turbine numbers $m_i = 15$ and $m_i = 70$, detailed in Tables 6 and 7. These tables report the mean and standard deviation of the conversion efficiency η for each algorithm under different wind and farm conditions. With fewer turbines, chaotic map-coded algorithms demonstrate marked superiority in P_1 and P_3 conditions. As the turbine count increases, their advantage in these conditions diminishes but remains above that of the traditional algorithms. Conversely, in P_4 conditions, chaotic map-coded algorithms initially show no benefit with a small turbine count, yet their advantage becomes apparent with larger turbine numbers. In the P_2 condition, CGLPSO and CSHADE do not outperform GLPSO and SHADE and sometimes exhibit reduced performance, whereas CSIS maintains superior performance.

Figure 11 displays Box-and-whisker plots of conversion efficiency for each algorithm across 13 wind farms, with turbine numbers $m_i = 15$ and $m_i = 70$. In these plots, Group 1 represents chaotic map-coded algorithms, and Group 2 consists of traditional algorithms. The plots reveal that Group 1 generally achieves a significantly better mean conversion efficiency in P_1 and P_3 conditions, no substantial advantage in P_4 , and an inferior mean performance in P_2 conditions.

Table 6 η comparison under 15 wind turbines

$m_t=15$	GLPSO		CGLPSO		SHADE		CSHADE		SIS		CSIS	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
P_1												
L0	0.968	0.008	0.979	0.001	+	0.972	0.003	0.979	0.000	+	0.963	0.005
L1	0.956	0.008	0.963	0.004	+	0.962	0.003	0.963	0.003	+	0.953	0.006
L2	0.974	0.004	0.979	0.000	+	0.975	0.002	0.979	0.000	+	0.968	0.004
L3	0.948	0.008	0.962	0.004	+	0.949	0.004	0.965	0.001	+	0.934	0.006
L4	0.945	0.010	0.962	0.003	+	0.948	0.005	0.964	0.001	+	0.932	0.007
L5	0.974	0.003	0.978	0.002	+	0.976	0.001	0.979	0.000	+	0.966	0.005
L6	0.959	0.010	0.966	0.008	+	0.966	0.004	0.967	0.005	=	0.956	0.007
L7	0.963	0.007	0.972	0.002	+	0.967	0.002	0.973	0.001	+	0.958	0.006
L8	0.972	0.005	0.979	0.001	+	0.975	0.002	0.979	0.000	+	0.966	0.005
L9	0.959	0.007	0.971	0.004	+	0.961	0.003	0.972	0.000	+	0.949	0.005
L10	0.956	0.010	0.970	0.003	+	0.960	0.003	0.972	0.000	+	0.949	0.006
L11	0.972	0.005	0.979	0.001	+	0.975	0.001	0.979	0.001	+	0.965	0.005
L12	0.964	0.007	0.971	0.005	+	0.967	0.003	0.974	0.003	+	0.959	0.006
W/T/L	13/0/0					12/1/0					13/0/0	
P_o	0.0002					0.0002					0.0002	
P_c	0.0029					0.0029					0.0029	
P_2												
L0	0.956	0.010	0.939	0.010	-	0.958	0.004	0.939	0.006	-	0.926	0.008
L1	0.930	0.008	0.922	0.010	-	0.936	0.005	0.917	0.007	-	0.901	0.008
L2	0.933	0.010	0.931	0.009	=	0.945	0.004	0.929	0.006	-	0.911	0.007
L3	0.921	0.009	0.919	0.008	=	0.934	0.005	0.917	0.007	-	0.901	0.007
L4	0.929	0.010	0.927	0.008	=	0.939	0.005	0.926	0.005	-	0.911	0.007
L5	0.933	0.009	0.934	0.008	=	0.946	0.005	0.935	0.005	-	0.918	0.008

Table 6 (continued)

P_2	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
L6	0.935	0.014	0.938	0.011	=	0.951	0.006	0.935	0.008	-	0.917	0.007
L7	0.945	0.009	0.930	0.009	-	0.949	0.004	0.932	0.007	-	0.915	0.008
L8	0.942	0.011	0.935	0.008	-	0.951	0.004	0.938	0.006	-	0.921	0.007
L9	0.933	0.011	0.932	0.009	=	0.947	0.004	0.931	0.006	-	0.915	0.008
L10	0.936	0.011	0.937	0.009	=	0.949	0.004	0.933	0.005	-	0.921	0.008
L11	0.939	0.009	0.939	0.010	=	0.953	0.004	0.940	0.005	-	0.926	0.007
L12	0.940	0.014	0.940	0.010	=	0.954	0.005	0.939	0.007	-	0.920	0.007
W/T/L	0.9/4					0.0/13					13/0/0	
P_o	0.0942					0.0002					0.0002	
P_c	0.0029					0.0029					0.0029	
P_3	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
L0	0.972	0.009	0.994	0.000	+	0.976	0.004	0.994	0.000	+	0.960	0.006
L1	0.967	0.010	0.987	0.002	+	0.969	0.004	0.987	0.001	+	0.952	0.006
L2	0.980	0.008	0.994	0.000	+	0.983	0.003	0.994	0.000	+	0.966	0.005
L3	0.967	0.009	0.987	0.005	+	0.969	0.004	0.989	0.002	+	0.946	0.005
L4	0.966	0.008	0.988	0.001	+	0.970	0.004	0.989	0.001	+	0.955	0.005
L5	0.982	0.005	0.994	0.000	+	0.985	0.004	0.994	0.000	+	0.964	0.006
L6	0.958	0.008	0.982	0.007	+	0.965	0.004	0.987	0.001	+	0.950	0.006
L7	0.968	0.010	0.991	0.001	+	0.973	0.004	0.992	0.000	+	0.957	0.005
L8	0.976	0.009	0.994	0.000	+	0.980	0.004	0.994	0.000	+	0.962	0.005
L9	0.970	0.009	0.991	0.001	+	0.973	0.004	0.992	0.000	+	0.954	0.004
L10	0.970	0.008	0.991	0.001	+	0.973	0.005	0.992	0.000	+	0.956	0.006
L11	0.978	0.008	0.994	0.000	+	0.981	0.004	0.994	0.000	+	0.961	0.004
L12	0.963	0.009	0.987	0.002	+	0.968	0.004	0.989	0.001	+	0.953	0.006

Table 6 (continued)

P_3	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
W/T/L	13/0/0		13/0/0		13/0/0		13/0/0		13/0/0	
P_o	0.0002		0.0002		0.0002		0.0002		0.0002	
P_c	0.0029		0.0029		0.0029		0.0029		0.0029	
P_4	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
L0	0.947	0.009	0.937	0.006	–		0.950	0.003	0.937	0.005
L1	0.935	0.011	0.928	0.006	–		0.940	0.004	0.926	0.005
L2	0.944	0.008	0.940	0.006	–		0.953	0.004	0.939	0.004
L3	0.929	0.008	0.928	0.007	=		0.939	0.005	0.927	0.005
L4	0.940	0.007	0.939	0.008	=		0.950	0.004	0.939	0.004
L5	0.942	0.007	0.940	0.006	=		0.951	0.004	0.942	0.005
L6	0.932	0.008	0.933	0.009	=		0.943	0.006	0.930	0.005
L7	0.940	0.011	0.933	0.007	–		0.945	0.004	0.934	0.006
L8	0.943	0.007	0.941	0.007	=		0.953	0.004	0.939	0.004
L9	0.937	0.008	0.934	0.007	–		0.945	0.004	0.933	0.004
L10	0.941	0.008	0.939	0.006	=		0.950	0.003	0.939	0.005
L11	0.941	0.006	0.942	0.008	=		0.951	0.004	0.941	0.005
L12	0.935	0.009	0.937	0.007	=		0.946	0.004	0.934	0.004
W/T/L	08/5		08/5		00/13		00/13		12/1/0	
P_o	0.0134		0.0134		0.0002		0.0002		0.0002	
P_c	0.0940		0.0940		0.0029		0.0029		0.0029	

Table 7 η comparison under 70 wind turbines

$m_t=70$	GLPSO		CGLPSO		SHADE		CSHADE		SIS		CSIS		
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	
P_1													
L0	0.418	0.004	0.426	0.003	+	0.417	0.002	0.427	0.002	+	0.405	0.003	+
L1	0.376	0.003	0.375	0.003	=	0.375	0.001	0.375	0.001	=	0.364	0.002	+
L2	0.420	0.004	0.423	0.006	+	0.420	0.002	0.422	0.003	+	0.411	0.002	+
L3	0.344	0.002	0.344	0.003	=	0.346	0.001	0.345	0.002	-	0.335	0.002	+
L4	0.344	0.002	0.345	0.002	=	0.346	0.001	0.344	0.001	-	0.336	0.002	+
L5	0.412	0.003	0.411	0.005	=	0.411	0.002	0.414	0.002	+	0.396	0.002	+
L6	0.368	0.003	0.370	0.004	+	0.372	0.001	0.369	0.002	-	0.363	0.002	+
L7	0.398	0.004	0.404	0.003	+	0.397	0.002	0.404	0.001	+	0.385	0.003	+
L8	0.421	0.003	0.425	0.005	+	0.419	0.002	0.425	0.002	+	0.408	0.002	+
L9	0.383	0.004	0.387	0.002	+	0.383	0.002	0.387	0.002	+	0.371	0.002	+
LL10	0.384	0.003	0.386	0.003	+	0.383	0.002	0.386	0.002	+	0.371	0.002	+
LL11	0.415	0.004	0.417	0.005	+	0.414	0.002	0.420	0.002	+	0.400	0.002	+
LL12	0.393	0.004	0.396	0.003	+	0.394	0.002	0.395	0.002	+	0.385	0.003	+
W/T/L	9/4/0					9/1/3					13/0/0		
P_o	0.0017					0.0398					0.0002		
P_c	0.0017					0.0398					0.0005		
P_2	Mean	Std	Mean	Std		Mean	Std	Mean	Std		Mean	Std	
L0	0.396	0.003	0.394	0.003	-	0.400	0.001	0.394	0.002	-	0.390	0.002	+
L1	0.343	0.002	0.340	0.002	-	0.345	0.001	0.339	0.001	-	0.338	0.002	=
L2	0.365	0.002	0.363	0.001	-	0.368	0.001	0.362	0.001	-	0.361	0.001	+
L3	0.342	0.002	0.341	0.002	-	0.344	0.001	0.339	0.002	-	0.338	0.001	=
L4	0.365	0.002	0.363	0.002	-	0.368	0.001	0.363	0.001	-	0.361	0.001	+
L5	0.387	0.002	0.385	0.002	-	0.390	0.001	0.384	0.001	-	0.383	0.001	=

Table 7 (continued)

P_2	Mean	Std	Mean	Std		Mean	Std	Mean	Std	Mean	Std	Mean	Std	
L6	0.357	0.002	0.357	0.002	=	0.360	0.001	0.355	0.001	-	0.353	0.001	0.002	+
L7	0.371	0.002	0.368	0.002	-	0.373	0.001	0.366	0.001	-	0.365	0.001	0.001	=
L8	0.382	0.003	0.380	0.002	-	0.384	0.001	0.378	0.001	-	0.376	0.001	0.001	+
L9	0.369	0.002	0.368	0.002	-	0.373	0.001	0.366	0.002	-	0.364	0.001	0.002	+
L10	0.382	0.003	0.380	0.002	-	0.385	0.001	0.379	0.001	-	0.376	0.001	0.002	+
L11	0.393	0.003	0.391	0.002	-	0.397	0.001	0.390	0.001	-	0.388	0.001	0.001	+
L12	0.380	0.003	0.379	0.003	=	0.385	0.001	0.378	0.002	-	0.375	0.001	0.002	+
W/T/L	0/2/11					0/0/13					9/4/0			
P_o	0.0002					0.0002					0.0005			
P_c	0.0029					0.0005					0.0010			
P_3	Mean	Std	Mean	Std		Mean	Std	Mean	Std	Mean	Std	Mean	Std	
L0	0.635	0.006	0.647	0.007	+	0.636	0.003	0.650	0.003	+	0.620	0.003	0.005	+
L1	0.599	0.006	0.593	0.004	-	0.601	0.002	0.591	0.004	-	0.584	0.002	0.003	+
L2	0.639	0.005	0.646	0.009	+	0.640	0.002	0.642	0.006	+	0.626	0.003	0.004	+
L3	0.597	0.005	0.599	0.004	+	0.601	0.003	0.597	0.003	-	0.586	0.003	0.003	+
L4	0.628	0.004	0.629	0.004	=	0.629	0.002	0.626	0.003	-	0.615	0.002	0.003	+
L5	0.643	0.005	0.641	0.007	=	0.644	0.003	0.641	0.005	-	0.632	0.003	0.004	+
L6	0.592	0.005	0.592	0.006	=	0.597	0.002	0.591	0.004	-	0.582	0.003	0.003	+
L7	0.619	0.007	0.619	0.006	=	0.619	0.003	0.620	0.003	+	0.602	0.003	0.004	+
L8	0.637	0.005	0.646	0.008	+	0.638	0.003	0.646	0.004	+	0.625	0.003	0.004	+
L9	0.617	0.006	0.623	0.006	+	0.619	0.003	0.622	0.003	+	0.603	0.003	0.004	+
L10	0.631	0.005	0.634	0.006	+	0.632	0.003	0.634	0.003	+	0.616	0.002	0.004	+
L11	0.640	0.005	0.643	0.008	+	0.641	0.003	0.645	0.003	+	0.626	0.003	0.005	+
L12	0.604	0.005	0.607	0.005	+	0.606	0.003	0.605	0.003	-	0.595	0.003	0.003	+

Table 7 (continued)

P_3	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	
W/T/L	8/4/1		7/0/6		13/0/0								
P_o	0.0215		0.8926		0.0002								
P_c	0.0215		0.8926		0.0005								
P_4	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	
L0	0.611	0.005	0.621	0.004	+	0.614	0.002	0.620	0.002	+	0.600	0.003	+
L1	0.577	0.005	0.572	0.003	-	0.579	0.002	0.570	0.002	-	0.565	0.002	+
L2	0.608	0.004	0.611	0.005	+	0.611	0.003	0.610	0.003	=	0.599	0.002	+
L3	0.576	0.004	0.581	0.006	+	0.579	0.003	0.580	0.002	+	0.565	0.002	+
L4	0.608	0.003	0.612	0.005	+	0.611	0.002	0.611	0.002	=	0.599	0.002	+
L5	0.619	0.003	0.621	0.005	+	0.621	0.002	0.619	0.003	-	0.609	0.002	+
L6	0.565	0.004	0.568	0.005	+	0.569	0.002	0.567	0.002	-	0.559	0.002	+
L7	0.594	0.004	0.599	0.005	+	0.597	0.002	0.600	0.002	+	0.583	0.002	+
L8	0.608	0.003	0.616	0.005	+	0.612	0.002	0.616	0.003	+	0.598	0.002	+
L9	0.593	0.004	0.602	0.004	+	0.597	0.002	0.601	0.002	+	0.583	0.003	+
L10	0.611	0.005	0.616	0.004	+	0.612	0.002	0.615	0.002	+	0.599	0.002	+
L11	0.617	0.004	0.621	0.005	+	0.619	0.002	0.620	0.003	+	0.605	0.003	+
L12	0.584	0.005	0.590	0.006	+	0.588	0.003	0.590	0.002	+	0.577	0.002	+
W/T/L	12/0/1		8/2/3								13/0/0		
P_o	0.0034		0.3396								0.0002		
P_c	0.0376		0.6792								0.0005		

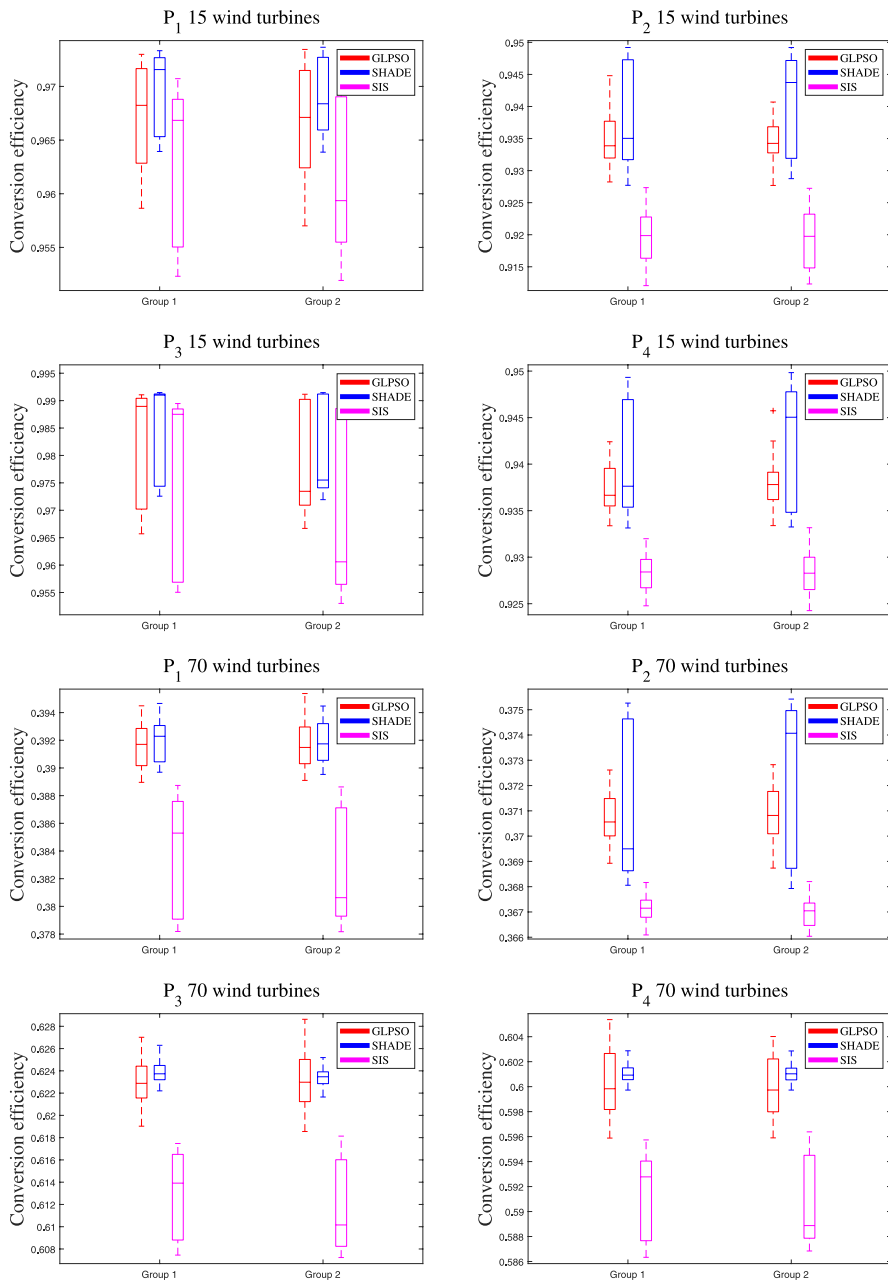


Fig. 11 The Box-and-whisker diagrams of the conversion efficiency. Group 1 in the figure is CGLPSO, CSHADE, and CSIS using chaotic transcription; group 2 is the traditional algorithms GLPSO, SHADE, and SIS

Table 8 The maximum $E(m_i, R_i)$ of 13 wind farms under four wind conditions

	SHADE				GLPSO				SIS			
	Total output		Number of turbines		Total output		Number of turbines		Total output		Number of turbines	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
P_1												
L0	18,866.90	139.55	63.43	3.05	18,372.05	318.15	71.92	6.06	18,288.34	151.18	74.18	8.26
L1	16,666.56	121.54	61.94	3.99	16,465.55	190.04	64.90	4.52	16,230.04	112.57	67.10	7.39
L2	18,544.43	220.03	66.69	4.70	18,320.14	267.89	71.94	4.58	18,240.32	115.73	74.33	6.61
L3	15,735.51	94.00	53.75	2.76	15,376.35	298.21	59.43	5.30	15,261.49	149.83	60.94	6.63
L4	15,677.13	83.15	54.61	3.16	15,365.04	222.19	59.45	4.97	15,188.28	141.73	61.94	6.25
L5	18,351.96	183.58	62.04	3.56	17,907.04	365.86	68.71	6.50	17,747.66	182.45	69.16	6.57
L6	16,437.18	178.88	61.31	3.96	16,277.16	236.39	65.08	5.66	16,166.54	155.96	66.94	7.77
L7	17,846.23	119.39	64.04	2.51	17,563.58	365.99	67.31	4.80	17,344.90	134.17	71.98	8.82
L8	18,649.62	212.36	66.10	4.83	18,359.20	306.67	72.73	5.65	18,318.14	158.33	73.96	8.73
L9	17,276.48	99.62	58.61	3.46	16,884.51	333.88	65.71	6.27	16,812.46	186.56	69.27	6.90
L10	17,261.93	124.65	58.73	4.07	16,827.38	260.52	66.84	5.35	16,716.47	147.86	69.31	7.97
L11	18,559.06	114.25	62.61	4.20	18,213.25	306.94	68.90	6.20	17,981.87	154.47	70.84	7.15
L12	17,464.64	154.27	64.27	3.84	17,259.09	212.45	67.08	6.08	17,051.84	113.85	68.73	7.29
AO	284.84				256.54				246.30			
P_0	–				0.0002				0.0002			
P_c	–				0.0010				0.0010			
P_2												
L0	17,289.00	77.79	73.02	2.98	17,346.62	121.52	72.96	5.19	17,143.51	63.48	75.37	5.50
L1	14,975.30	58.81	63.65	2.70	15,002.62	100.41	64.65	4.30	14,895.03	68.66	64.75	5.73
L2	15,943.44	54.25	67.45	2.93	15,948.18	96.88	70.14	4.06	15,887.11	61.68	68.43	6.28

Table 8 (continued)

	SHADE			GLPSO			SIS							
	Total output			Number of turbines			Total output			Number of turbines				
	Mean	Std		Mean	Std		Mean	Std		Mean	Std			
L3	14,979.70	58.97		63.90	2.91		14,999.92	96.69	64.12	4.19	14,889.89	73.90	65.08	6.40
L4	15,947.33	53.95		67.65	3.02		15,981.83	86.96	67.94	4.40	15,866.46	63.78	69.47	5.83
L5	16,909.29	67.32		72.10	3.21		16,916.34	103.25	72.90	4.27	16,815.27	49.64	74.49	5.14
L6	15,680.26	78.47		65.80	3.93		15,711.19	118.78	66.92	4.28	15,587.52	72.66	66.24	3.99
L7	16,119.16	70.88		68.88	3.13		16,151.54	133.46	69.51	4.31	15,990.35	64.19	70.37	5.65
L8	16,617.86	66.44		70.08	3.89		16,678.62	91.37	71.53	4.81	16,535.96	73.42	71.24	6.64
L9	16,108.30	75.04		67.88	3.79		16,157.45	135.17	69.00	4.43	16,029.90	88.97	68.18	5.32
L10	16,638.53	60.21		70.88	2.92		16,706.33	96.88	70.45	4.20	16,535.50	74.70	73.31	5.49
L11	17,153.79	75.43		72.94	3.49		17,169.48	101.59	74.25	4.94	17,047.38	68.17	75.88	5.11
L12	16,595.83	68.99		69.00	3.62		16,626.76	126.70	68.92	4.16	16,482.57	98.90	70.80	5.84
AO	236.17						234.03					229.54		
P_o	–						0.0002					0.0002		
P_c	–						0.0010					0.0010		
P_3														
L0	35,118.72	4.10		141.65	0.48		35,115.30	5.61	141.94	0.58	35,111.46	5.37	142.75	1.81
L1	30,376.55	4.28		117.71	0.54		30,373.08	7.43	118.04	0.77	30,368.95	6.36	119.22	1.08
L2	33,622.80	0.00		120.00	0.00		33,622.80	0.00	120.00	0.00	33,622.80	0.00	119.12	1.81
L3	30,675.35	0.00		120.00	0.00		30,675.48	0.72	119.88	0.59	30,675.35	0.00	119.45	1.22
L4	33,539.46	0.00		120.00	0.00		33,539.46	0.00	120.00	0.00	33,539.46	0.00	119.27	0.98
L5	34,604.48	0.00		120.00	0.00		34,604.48	0.00	120.00	0.00	34,604.48	0.00	119.33	1.16
L6	30,203.17	0.41		119.88	0.48		30,203.26	0.59	119.80	0.60	30,203.24	0.57	119.29	1.58

Table 8 (continued)

	SHADE			GLPSO			SIS		
	Number of turbines			Number of turbines			Number of turbines		
	Mean	Std		Mean	Std		Mean	Std	
	Total output			Total output			Total output		
	Mean	Std		Mean	Std		Mean	Std	
L7	32,748.59	3.85		32,743.53	8.72		32,740.41	5.45	
L8	34,558.46	0.99		34,558.34	0.50		34,558.62	1.06	
L9	32,903.55	2.84		32,900.04	4.73		32,899.44	3.57	
L10	34,244.78	7.95		34,240.20	11.23		34,231.91	10.59	
L11	35,126.22	0.00		35,126.22	0.00		35,126.22	0.00	
L12	32,157.34	3.94		32,151.41	6.84		32,150.57	6.70	
AO	261.80			261.42			261.06		
P_o	—			0.0273			0.0391		
P_c	—			0.0313			0.0391		
P_4									
L0	27,573.41	4.10		27,570.93	6.02		27,568.96	5.47	
L1	23,979.87	2.14		23,977.42	5.17		23,975.23	4.00	
L2	26,379.50	0.00		26,379.50	0.00		26,379.50	0.00	
L3	23,980.19	1.96		23,976.56	5.17		23,976.70	4.98	
L4	26,379.50	0.00		26,379.50	0.00		26,379.50	0.00	
L5	27,155.35	0.00		27,155.35	0.00		27,155.35	0.00	
L6	23,586.42	3.44		23,582.25	5.10		23,581.98	3.48	
L7	25,780.63	3.75		25,777.36	4.60		25,774.96	5.89	
L8	26,992.87	0.00		26,992.87	0.00		26,992.87	0.00	
L9	25,780.97	3.90		25,776.81	6.49		25,775.33	5.81	
L10	26,992.87	0.00		26,992.87	0.00		26,992.87	0.00	

Table 8 (continued)

	SHADE				GLPSO				SIS			
	Total output		Number of turbines		Total output		Number of turbines		Total output		Number of turbines	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
L11	27,541.63	0.00	132.00	0.00	27,541.63	0.00	132.00	0.00	27,541.63	0.00	131.71	0.61
L12	25,278.05	4.09	129.88	0.38	25,271.70	6.24	130.20	0.80	25,268.98	4.16	131.39	1.06
AO	205.70				205.34				204.72			
P_o	–				0.0156				0.0156			
P_c	–				0.0313				0.0313			

5.3.2 Wind farm layout optimization with variable turbine count

The optimization outcomes in this section reveal that metaheuristic algorithms encoded with chaotic maps are capable of effectively handling metameric variable-length problems. Table 8 presents the maximum total power output, denoted as $E(m_i, R_i)$. Alongside this, the table records the corresponding number of turbines that achieve this maximum output. Notably, the CSHADE algorithm consistently identifies the maximum total output power in wind conditions P_1 , P_3 , and P_4 , while CGLPSO excels in the P_2 scenario. Furthermore, the turbine count required for each wind farm, when optimized by CSHADE, is found to be the lowest across all wind conditions. To account for multiple comparisons across the four wind conditions (P_1 – P_4), we applied the Holm–Bonferroni correction to the p -values obtained from the Wilcoxon signed-rank tests. The corrected values p_c are reported in Table 8 for both SHADE vs. GLPSO and SHADE vs. SIS. The results indicate that in both cases, the performance differences remain statistically significant after correction ($p_c < 0.05$).

Additionally, the term ‘AO’ in the table refers to the average output power of individual turbines. It is observed that turbines optimized by CSHADE exhibit the highest average output power. It is crucial to note that traditional metaheuristics struggle with optimizing for the maximum total output $E(m_i, R_i)$, primarily due to the varying number of turbines in different layout schemes, which introduces inconsistencies in the dimensionality among individuals.

Figure 12 illustrates the convergence curves of the maximum total output and the turbine count for each algorithm across iterations. In these curves, a higher total maximum output is preferable, and a lower number of turbines is advantageous. The data reveals that, except in the P_2 condition, CSHADE consistently achieves the highest total maximum output and maintains the lowest turbine count. This underscores CSHADE’s effectiveness in maximizing wind farm output under turbulent wind conditions.

5.3.3 Comparison with variable length genetic algorithm

The objective of this section is to assess whether metaheuristic algorithms encoded with chaotic maps outperform mainstream VGA. In this section, CSHADE, which demonstrated the best performance in previous experiments, is selected for comparison with VGA. The comparative results are illustrated in Table 9. As shown in the table, CSHADE underperforms VGA under the first three wind scenarios (P_1 – P_3), with statistically significant differences in total energy output across all 13 wind farm layouts ($p_c < 0.05$). However, as the wind scenarios become increasingly complex—from a single wind direction in P_1 to twelve directions and variable wind speeds in P_4 —the performance gap between CSHADE and VGA narrows. In the most complex scenario P_4 , the difference is no longer statistically significant ($p_c = 0.2969$), indicating that CSHADE performs comparably to VGA under diverse and dynamic wind conditions. This suggests that chaotic map-based encodings may possess potential that warrants further investigation, especially under highly variable environmental conditions, even though their superiority over traditional methods has not yet been established.

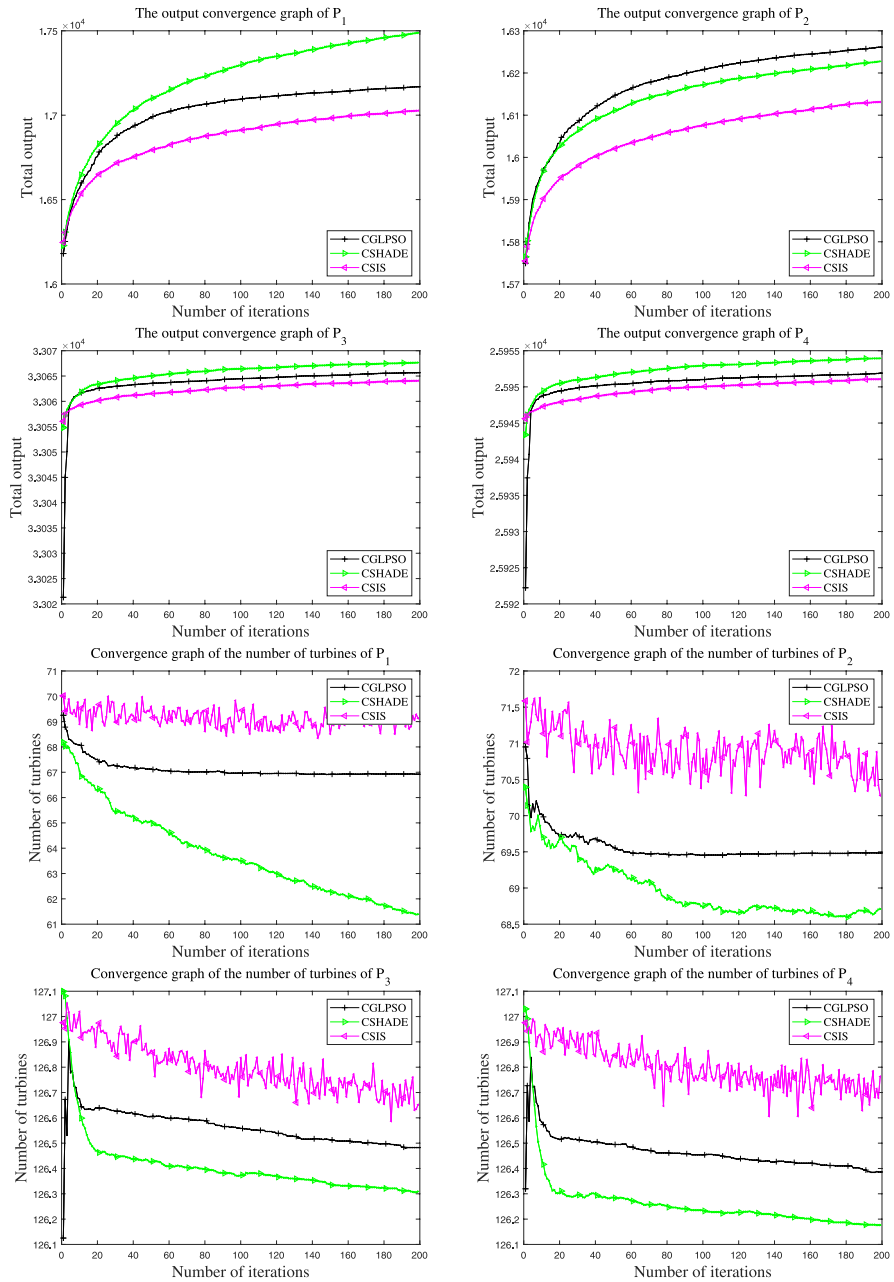


Fig. 12 The convergence curves of the maximum total output and the number of turbines

Table 9 The maximum $E(m_i, R_i)$ of 13 wind farms under four wind conditions

	VGA				CSHADE				
	Total output		Number of turbines		Total output		Number of turbines		
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	
P_1									
L0	19,227.38	156.20	71.24	4.95	18,866.90	139.55	63.43	3.05	+
L1	17,254.41	145.89	63.22	4.17	16,666.56	121.54	61.94	3.99	+
L2	19,224.24	164.02	71.16	4.27	18,544.43	220.03	66.69	4.70	+
L3	16,065.45	138.06	58.73	4.55	15,735.51	94.00	53.75	2.76	+
L4	16,060.06	106.26	59.00	4.09	15,677.13	83.15	54.61	3.16	+
L5	18,745.67	144.99	66.47	4.12	18,351.96	183.58	62.04	3.56	+
L6	17,199.42	164.29	63.29	4.54	16,437.18	178.88	61.31	3.96	+
L7	18,249.84	128.98	66.71	4.23	17,846.23	119.39	64.04	2.51	+
L8	19,269.55	142.90	70.92	4.42	18,649.62	212.36	66.10	4.83	+
L9	17,622.20	151.76	65.35	4.47	17,276.48	99.62	58.61	3.46	+
L10	17,632.75	159.79	64.88	5.15	17,261.93	124.65	58.73	4.07	+
L11	18,929.36	164.77	69.84	4.75	18,559.06	114.25	62.61	4.20	+
L12	18,207.37	134.91	66.69	4.15	17,464.64	154.27	64.27	3.84	+
W/T/L					13/0/0				
p_o	0.0002								
p_c	0.0010								
P_2									
L0	19,227.38	156.20	77.96	4.50	17,289.00	77.79	73.02	2.98	+
L1	17,254.41	145.89	67.25	3.93	14,975.30	58.81	63.65	2.70	+
L2	19,224.24	164.02	71.24	3.97	15,943.44	54.25	67.45	2.93	+
L3	16,065.45	138.06	65.73	3.87	14,979.70	58.97	63.90	2.91	+
L4	16,060.06	106.26	69.78	4.16	15,947.33	53.95	67.65	3.02	+
L5	18,745.67	144.99	75.88	3.66	16,909.29	67.32	72.10	3.21	+
L6	17,199.42	164.29	67.96	3.87	15,680.26	78.47	65.80	3.93	+
L7	18,249.84	128.98	72.06	3.88	16,119.16	70.88	68.88	3.13	+
L8	19,269.55	142.90	73.90	3.77	16,617.86	66.44	70.08	3.89	+
L9	17,622.20	151.76	72.08	3.60	16,108.30	75.04	67.88	3.79	+
L10	17,632.75	159.79	73.47	4.00	16,638.53	60.21	70.88	2.92	+
L11	18,929.36	164.77	76.92	4.11	17,153.79	75.43	72.94	3.49	+
L12	18,207.37	134.91	71.88	4.69	16,595.83	68.99	69.00	3.62	+
W/T/L					13/0/0				
p_o	0.0002								
p_c	0.0010								
P_3									
L0	35,134.39	20.81	140.61	1.42	35,118.72	4.10	141.65	0.48	+
L1	30,384.93	18.96	117.18	1.56	30,376.55	4.28	117.71	0.54	+
L2	33,622.80	0.00	120.00	0.00	33,622.80	0.00	120.00	0.00	=
L3	30,675.39	0.29	119.94	0.42	30,675.35	0.00	120.00	0.00	=

Table 9 (continued)

	VGA				CSHADE				
	Total output		Number of turbines		Total output		Number of turbines		
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	
L4	33,539.46	0.00	120.00	0.00	33,539.46	0.00	120.00	0.00	=
L5	34,604.48	0.00	120.00	0.00	34,604.48	0.00	120.00	0.00	=
L6	30,203.24	1.19	119.92	0.56	30,203.17	0.41	119.88	0.48	=
L7	32,760.34	17.61	128.82	1.37	32,748.59	3.85	129.63	0.56	+
L8	34,558.60	1.66	131.88	0.62	34,558.46	0.99	131.92	0.39	=
L9	32,907.19	11.87	129.65	1.38	32,903.55	2.84	129.90	0.41	=
L10	34,246.97	27.70	129.67	1.03	34,244.78	7.95	129.39	0.60	=
L11	35,126.22	0.00	132.00	0.00	35,126.22	0.00	132.00	0.00	=
L12	32,160.49	12.95	129.65	1.25	32,157.34	3.94	129.94	0.31	+
W/T/L					4/9/0				
p_o	0.0039								
p_c	0.0078								
P_4									
L0	27,583.91	14.37	140.86	1.13	27,573.41	4.10	141.27	0.67	+
L1	23,979.53	9.43	118.02	1.14	23,979.87	2.14	117.96	0.20	=
L2	26,379.50	0.00	120.00	0.00	26,379.50	0.00	120.00	0.00	=
L3	23,977.99	8.29	118.33	0.91	23,980.19	1.96	117.88	0.33	−
L4	26,379.50	0.00	120.00	0.00	26,379.50	0.00	120.00	0.00	=
L5	27,155.35	0.00	120.00	0.00	27,155.35	0.00	120.00	0.00	=
L6	23,585.80	8.87	118.12	1.11	23,586.42	3.44	117.98	0.32	=
L7	25,785.03	12.31	129.45	1.08	25,780.63	3.75	129.67	0.52	=
L8	26,992.87	0.00	132.00	0.00	26,992.87	0.00	132.00	0.00	=
L9	25,783.07	9.79	129.57	0.92	25,780.97	3.90	129.65	0.52	=
L10	26,992.87	0.00	132.00	0.00	26,992.87	0.00	132.00	0.00	=
L11	27,541.63	0.00	132.00	0.00	27,541.63	0.00	132.00	0.00	=
L12	25,286.11	12.07	129.14	1.00	25,278.05	4.09	129.88	0.38	+
W/T/L					2/10/1				
p_o	0.2969								
p_c	0.2969								

Notably, in the P_4 scenario, there is even a 2/10/1 W/T/L occurrence, indicating that CSHADE has significantly outperformed VGA in one type of land condition. Figure 13 also provides an analysis of energy output and the number of turbines for two different locations, $L0$ and $L3$, under the P_4 wind condition. It is clearly observed that in the case of $L3$, the wind farm layout optimized by CSHADE achieves greater energy output with fewer turbines. In this section, we conclude that although the overall performance of the chaotic mapping encoded metaheuristic is weaker than VGA on variable-length WFLOP, as the complexity of wind conditions

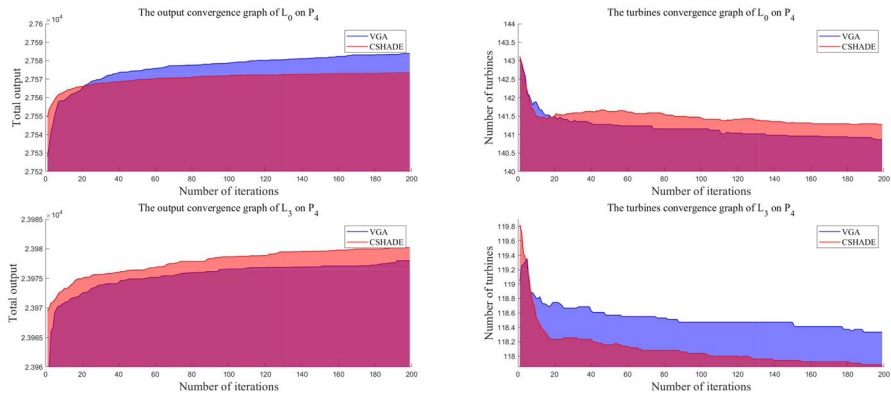


Fig. 13 The convergence curves of the maximum total output and the number of turbines

increases, there is a trend of CSHADE's performance gradually catching up to and even surpassing that of VGA.

6 Discussion and conclusion

This study proposes an efficient variable-length technique that enables metaheuristic algorithms to solve previously intractable problems. For example, in WFLOP, if there are 100 possible turbine placement locations, traditional fixed-length metaheuristic methods would require predefining the number of turbines as 1, 2, 3, ..., 100. In other words, the traditional approach would need to run 100 separate optimizations to determine the optimal number and placement of turbines. In contrast, a variable-length technique can accomplish this task in a single optimization run, significantly reducing computational costs while improving solution quality. Our method introduces a novel variable-length technique: chaotic transcription. This technique offers several advantages. First, chaotic maps exhibit extreme sensitivity to initial conditions, where slight variations in the initial state lead to exponentially diverging trajectories over time. Moreover, over a sufficiently long time scale, the trajectory of a chaotic map can cover the entire state space (or traverse an invariant subset), ensuring uniform exploration without being constrained by periodic structures. This property enhances population diversity in the algorithm and prevents premature convergence to local optima. Additionally, although chaotic behavior appears random, it is governed by a deterministic dynamical system. Given an initial state, the future states are fully determined by the mapping equation rather than by stochastic processes. This characteristic allows individuals in an algorithm, regardless of the dimensionality of the solution space, to be compressed into a single deterministic initial value. Such a compression mechanism enables simultaneous exploration across multiple solution spaces with varying dimensionalities. Based on these properties, we speculate that metaheuristic algorithms incorporating chaotic transcription may exhibit strong performance in large-scale, ultra-high-dimensional,

and complex continuous spaces. This advantage could be particularly beneficial for evolving large-scale parameters in neural network models.

This study also has certain limitations. Metaheuristics aim to find acceptable solutions with limited computational power. To achieve this goal, either the algorithm's exploitation capability in local search needs to be enhanced, or its exploration capability in global search needs to be improved. Chaotic maps focus on enhancing the algorithm's exploration capability, aiming to search as large a space as possible with minimal resources. Specifically, chaotic transcription compresses information in the solution space using chaotic maps, combining multiple solution spaces into the same dimension. This combination sacrifices some precision in local search but significantly increases the diversity of the algorithm's distribution in the solution space, even allowing the algorithm to sample in solution spaces of different dimensions, thereby enabling the exploration of larger solution spaces. This may also explain why metaheuristics encoded with chaotic maps perform better in continuous spaces than in discrete spaces. Clearly, continuous solution spaces are typically more complex as they involve handling an infinite number of states, which necessitates a stronger search capability from the algorithm to traverse the solution space.

Furthermore, it is particularly noteworthy that, although metaheuristics encoded with chaotic maps may underperform in comparison to traditional variable-length genetic algorithms on discrete problems like wind farm layout optimization, they significantly outshine variable-length genetic algorithms in continuous problems such as dendritic neuron structure optimization. This underscores the significance of our research: employing chaotic map encoding techniques to enhance diversity in variable-length metaheuristics contributes to the development of efficient optimization algorithms for specific metameric variable-length problems. Of course, future research will also be dedicated to continuously improving the performance of chaotic map encoding techniques across various problem types. In the next step, we will explore various chaotic maps, such as the Logistic and Gaussian maps for improving the performance of chaotic maps on metaheuristics.

Acknowledgements This research was partially supported by the Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant 19H01096, 21H05293, and 21H05298, and the Tokushima University Tenure-Track Faculty Development Support System, Tokushima University, Japan.

Author contributions H.Y. led the conceptualization, data curation, formal analysis, investigation, methodology, software development, validation, visualization, and the writing of the original draft, as well as project administration and funding acquisition. H.L. contributed to data curation, formal analysis, and methodology. Y.Y. participated in conceptualization, data curation, and formal analysis. C.N. and S.K. were responsible for validation and visualization. K.H. provided support for conceptualization and funding acquisition. Y.N. oversaw project administration. All authors reviewed the manuscript.

Data availability The code will appear online: <https://github.com/Hai-chuan-Yang/Chaos-Transcription>.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. I. Grossmann, R. Apap, B. Calfa, P. Garcia-Herreros, Q. Zhang, Mathematical programming techniques for optimization under uncertainty and their application in process systems engineering. *Theor. Found. Chem. Eng.* **51**(6), 893–909 (2017). <https://doi.org/10.1134/S0040579517060057>
2. K. Hussain, M.N.M. Salleh, S. Cheng, Y. Shi, Metaheuristic research: a comprehensive survey. *Artif. Intell. Rev.* **52**(4), 2191–2233 (2019). <https://doi.org/10.1007/s10462-017-9605-z>
3. M.R. Garey, D.S. Johnson, L. Stockmeyer, Some simplified np-complete problems, in *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing* (1974), pp. 47–63. <https://doi.org/10.1145/800119.803884>
4. J.H. Holland, Genetic algorithms. *Sci. Am.* **267**(1), 66–73 (1992)
5. R. Storn, K. Price, Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J. Glob. Optim.* **11**(4), 341–359 (1997). <https://doi.org/10.1023/A:1008202821328>
6. R. Tanabe, A. Fukunaga, Success-history based parameter adaptation for differential evolution, in *2013 IEEE Congress on Evolutionary Computation* (2013), pp. 71–78. <https://doi.org/10.1109/CEC.2013.6557555>
7. S. Das, P.N. Suganthan, Differential evolution: a survey of the state-of-the-art. *IEEE Trans. Evol. Comput.* **15**(1), 4–31 (2011). <https://doi.org/10.1109/TEVC.2010.2059031>
8. R. Tanabe, A.S. Fukunaga, Improving the search performance of shade using linear population size reduction, in *2014 IEEE Congress on Evolutionary Computation (CEC)* (2014), pp. 1658–1665. <https://doi.org/10.1109/CEC.2014.6900380>
9. M. Dorigo, M. Birattari, T. Stutzle, Ant colony optimization. *IEEE Comput. Intell. Mag.* **1**(4), 28–39 (2006). <https://doi.org/10.1109/MCI.2006.329691>
10. J. Kennedy, R. Eberhart, Particle swarm optimization, in *Proceedings of ICNN'95—International Conference on Neural Networks*, vol. 4 (1995), pp. 1942–19484. <https://doi.org/10.1109/ICNN.1995.488968>
11. Y.-J. Gong, J.-J. Li, Y. Zhou, Y. Li, H.S.-H. Chung, Y.-H. Shi, J. Zhang, Genetic learning particle swarm optimization. *IEEE Trans. Cybern.* **46**(10), 2277–2290 (2016). <https://doi.org/10.1109/TCYB.2015.2475174>
12. H. Yang, Y. Yu, J. Cheng, Z. Lei, Z. Cai, Z. Zhang, S. Gao, An intelligent metaphor-free spatial information sampling algorithm for balancing exploitation and exploration. *Knowl. Based Syst.* (2022). <https://doi.org/10.1016/j.knosys.2022.109081>
13. H. Escobar-Cuevas, E. Cuevas, J. Lopez, M. Perez-Cisneros, Integration of metaheuristic operators through unstructured evolutive game theory approach: a novel hybrid methodology. *Evol. Intell.* **18**(11), 1–28 (2025). <https://doi.org/10.1007/s12065-024-00988-7>
14. E. Cuevas, M. Vásquez, K. Avila, A. Rodríguez, D. Zaldivar, Balancing individual and collective strategies: a new approach in metaheuristic optimization. *Math. Comput. Simul.* **227**, 322–346 (2025). <https://doi.org/10.1016/j.matcom.2024.08.004>
15. B.A. Rivera-Aguilar, E. Cuevas, D. Zaldivar, M.A. Pérez-Cisneros, A metaheuristic algorithm based on a radial basis function neural networks. *Neural Comput. Appl.* **36**, 22119–22147 (2024). <https://doi.org/10.1007/s00521-024-10372-2>
16. M.L. Ryerkerk, R.C. Averill, K. Deb, E.D. Goodman, Solving metameric variable-length optimization problems using genetic algorithms. *Genet. Program. Evolvable Mach.* **18**(2), 247–277 (2017). <https://doi.org/10.1007/s10710-016-9282-8>
17. C.-K. Ting, C.-N. Lee, H.-C. Chang, J.-S. Wu, Wireless heterogeneous transmitter placement using multiobjective variable-length genetic algorithm. *IEEE Trans. Syst. Man Cybern. Part B (Cybern.)* **39**(4), 945–958 (2009). <https://doi.org/10.1109/TSMCB.2008.2010951>
18. J.S. González, A.G. Rodríguez, J.C. Mora, M.B. Payán, J.R. Santos, Overall design optimization of wind farms. *Renew. Energy* **36**(7), 1973–1982 (2011). <https://doi.org/10.1016/j.renene.2010.10.034>
19. S. Gao, M. Zhou, Y. Wang, J. Cheng, H. Yachi, J. Wang, Dendritic neuron model with effective learning algorithms for classification, approximation, and prediction. *IEEE Trans. Neural Netw. Learn. Syst.* **30**(2), 601–614 (2019). <https://doi.org/10.1109/TNNLS.2018.2846646>
20. C. Kettle, J. Johnstone, T. Jowett, H. Arthur, W. Arthur, The pattern of segment formation, as revealed by engrailed expression, in a centipede with a variable number of segments. *Evol. Dev.* **5**(2), 198–207 (2003). <https://doi.org/10.1046/j.1525-142X.2003.03027.x>

21. M. Ryerkerk, R. Averill, K. Deb, E. Goodman, A survey of evolutionary algorithms using meta-meric representations. *Genet. Program. Evolvable Mach.* **20**(4), 441–478 (2019). <https://doi.org/10.1007/s10710-019-09356-2>
22. F. Rothlauf, *Representations for Genetic and Evolutionary Algorithms* (Springer, Berlin, Heidelberg, 2006), pp. 9–32. https://doi.org/10.1007/3-540-32444-5_2
23. M. Khodzhaiev, U. Reuter, Structural optimization of transmission towers using a novel genetic algorithm approach with a variable length genome. *Eng. Struct.* **240**, 112306 (2021). <https://doi.org/10.1016/j.engstruct.2021.112306>
24. R. Pal, T.D. Chaudhuri, S. Mukhopadhyay, Portfolio formation and optimization with continuous realignment: a suggested method for choosing the best portfolio of stocks using variable length NSGA-II. *Expert Syst. Appl.* **186**, 115732 (2021). <https://doi.org/10.1016/j.eswa.2021.115732>
25. M.-A. Tsompanas, L. Bull, A. Adamatzky, I. Balaz, Metameric representations on optimization of nano particle cancer treatment. *Biocybern. Biomed. Eng.* **41**(2), 352–361 (2021). <https://doi.org/10.1016/j.bbe.2021.02.002>
26. Y. Jiang, L. Tang, H. Liu, A. Zeng, A variable-length encoding genetic algorithm for incremental service composition in uncertain environments for cloud manufacturing. *Appl. Soft Comput.* **123**, 108902 (2022). <https://doi.org/10.1016/j.asoc.2022.108902>
27. Z. Su, P. Wang, J. Shen, Y. Li, Y. Zhang, E. Hu, Automatic fuzzy partitioning approach using variable string length artificial bee colony (VABC) algorithm. *Appl. Soft Comput.* **12**(11), 3421–3441 (2012). <https://doi.org/10.1016/j.asoc.2012.06.019>
28. A. Mohammadi, S.H. Zahir, S.M. Razavi, P.N. Suganthan, Design and modeling of adaptive IIR filtering systems using a weighted sum-variable length particle swarm optimization. *Appl. Soft Comput.* **109**, 107529 (2021). <https://doi.org/10.1016/j.asoc.2021.107529>
29. J. Huang, B. Xue, Y. Sun, M. Zhang, A flexible variable-length particle swarm optimization approach to convolutional neural network architecture design, in *2021 IEEE Congress on Evolutionary Computation (CEC)* (2021), pp. 934–941. <https://doi.org/10.1109/CEC45853.2021.9504716>
30. Y. Chen, V. Mahalec, Y. Chen, X. Liu, R. He, K. Sun, Reconfiguration of satellite orbit for cooperative observation using variable-size multi-objective differential evolution. *Eur. J. Oper. Res.* **242**(1), 10–20 (2015). <https://doi.org/10.1016/j.ejor.2014.09.025>
31. A. Ghosh, N.D. Jana, S. Mallik, Z. Zhao, Designing optimal convolutional neural network architecture using differential evolution algorithm. *Patterns* **3**(9), 100567 (2022)
32. B. Tran, B. Xue, M. Zhang, Variable-length particle swarm optimization for feature selection on high-dimensional classification. *IEEE Trans. Evol. Comput.* **23**(3), 473–487 (2019). <https://doi.org/10.1109/TEVC.2018.2869405>
33. K.O. Stanley, Compositional pattern producing networks: a novel abstraction of development. *Genet. Program. Evolvable Mach.* **8**, 131–162 (2007). <https://doi.org/10.1007/s10710-007-9028-8>
34. K.O. Stanley, J. Clune, J. Lehman, R. Miikkulainen, Designing neural networks through neuroevolution. *Nat. Mach. Intell.* **1**(1), 24–35 (2019). <https://doi.org/10.1038/s42256-018-0006-z>
35. F. Crick, Central dogma of molecular biology. *Nature* **227**(5258), 561–563 (1970). <https://doi.org/10.1038/227561a0>
36. M. Bizzarri, A. Cucina, F. Conti, F. D'Anselmi, Beyond the oncogene paradigm: understanding complexity in cancerogenesis. *Acta. Biotheor.* **56**(3), 173–196 (2008). <https://doi.org/10.1007/s10441-008-9047-8>
37. J.E. Skinner, Low-dimensional chaos in biological systems. *Bio/technology* **12**(6), 596–600 (1994). <https://doi.org/10.1038/nbt0694-596>
38. M.L. Heltberg, S. Krishna, M.H. Jensen, On chaotic dynamics in transcription factors and the associated effects in differential gene regulation. *Nat. Commun.* **10**(1), 71 (2019). <https://doi.org/10.1038/s41467-018-07932-1>
39. S. Gao, Y. Yu, Y. Wang, J. Wang, J. Cheng, M. Zhou, Chaotic local search-based differential evolution algorithms for optimization. *IEEE Trans. Syst. Man Cybern.: Syst.* **51**(6), 3954–3967 (2021). <https://doi.org/10.1109/TSMC.2019.2956121>
40. H. Yang, Y. Yang, Y. Zhang, C. Tang, K. Hashimoto, Y. Nagata, Chaotic map-coded evolutionary algorithms for dendritic neuron model optimization, in *2024 IEEE Congress on Evolutionary Computation (CEC)* (2024), pp. 1–8. <https://doi.org/10.1109/CEC60901.2024.10612087>
41. W.S. McCulloch, W. Pitts, A logical calculus of the ideas immanent in nervous activity. *Bull. Math. Biophys.* **5**(4), 115–133 (1943). <https://doi.org/10.1007/BF02478259>

42. Z. Li, F. Liu, W. Yang, S. Peng, J. Zhou, A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Trans. Neural Netw. Learn. Syst.* **33**(12), 6999–7019 (2022). <https://doi.org/10.1109/TNNLS.2021.3084827>
43. Y. Yu, X. Si, C. Hu, J. Zhang, A review of recurrent neural networks: Lstm cells and network architectures. *Neural Comput.* **31**(7), 1235–1270 (2019). https://doi.org/10.1162/neco_a_01199
44. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in *Advances in Neural Information Processing Systems*, vol. 30 (Curran Associates, Inc., 2017)
45. J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, *Bert: pre-training of deep bidirectional transformers for language understanding* (2018). [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
46. T.B. Brown, *Language models are few-shot learners* (2020). [arXiv:2005.14165](https://arxiv.org/abs/2005.14165)
47. E.M. Izhikevich, Simple model of spiking neurons. *IEEE Trans. Neural Netw.* **14**(6), 1569–1572 (2003). <https://doi.org/10.1109/TNN.2003.820440>
48. J. Sîma, Training a single sigmoidal neuron is hard. *Neural Comput.* **14**(11), 2709–2728 (2002). <https://doi.org/10.1162/089976602760408035>
49. Y. Zhang, Y. Yang, X. Li, Z. Yuan, Y. Todo, H. Yang, A dendritic neuron model optimized by meta-heuristics with a power-law-distributed population interaction network for financial time-series forecasting. *Mathematics* **11**(5), 1251 (2023). <https://doi.org/10.3390/math11051251>
50. Y. Yu, Z. Lei, Y. Wang, T. Zhang, C. Peng, S. Gao, Improving dendritic neuron model with dynamic scale-free network-based differential evolution. *IEEE/CAA J. Autom. Sin.* **9**(1), 99–110 (2022). <https://doi.org/10.1109/JAS.2021.1004284>
51. H. Yang, Y. Zhang, C. Zhang, W. Xia, Y. Yang, Z. Zhang, A hyperparameter self-evolving shade-based dendritic neuron model for classification. *Axioms* **12**(11), 1051 (2023). <https://doi.org/10.3390/axioms12111051>
52. M. Cheng, Y. Zhu, The state of the art of wind energy conversion systems and technologies: a review. *Energy Convers. Manag.* **88**, 332–347 (2014). <https://doi.org/10.1016/j.enconman.2014.08.037>
53. P. Sadorsky, Wind energy for sustainable development: driving factors and future outlook. *J. Clean. Prod.* **289**, 125779 (2021). <https://doi.org/10.1016/j.jclepro.2020.125779>
54. H. Chen, Y. Shi, X. Zhao, Investment in renewable energy resources, sustainable financial inclusion and energy efficiency: a case of us economy. *Resour. Policy* **77**, 102680 (2022). <https://doi.org/10.1016/j.resourpol.2022.102680>
55. A.P. Marugán, F.P.G. Márquez, J.M.P. Perez, D. Ruiz-Hernández, A survey of artificial neural network in wind energy systems. *Appl. Energy* **228**, 1822–1836 (2018). <https://doi.org/10.1016/j.apenergy.2018.07.084>
56. R. McKenna, S. Pfenninger, H. Heinrichs, J. Schmidt, I. Staffell, C. Bauer, K. Gruber, A.N. Hahmann, M. Jansen, M. Klingler et al., High-resolution large-scale onshore wind energy assessments: a review of potential definitions, methodologies and future research needs. *Renew. Energy* **182**, 659–684 (2022). <https://doi.org/10.1016/j.renene.2021.10.02>
57. S.R. Reddy, Wind farm layout optimization (WindFLO): an advanced framework for fast wind farm analysis and optimization. *Appl. Energy* **269**, 115090 (2020). <https://doi.org/10.1016/j.apenergy.2020.115090>
58. L. Parada, C. Herrera, P. Flores, V. Parada, Wind farm layout optimization using a Gaussian-based wake model. *Renew. Energy* **107**, 531–541 (2017). <https://doi.org/10.1016/j.renene.2017.02.017>
59. B.L. Du Pont, J. Cagan, An extended pattern search approach to wind farm layout optimization. *J. Mech. Des.* **134**(8), 081002 (2012). <https://doi.org/10.1115/1.4006997>
60. F. Bai, X. Ju, S. Wang, W. Zhou, F. Liu, Wind farm layout optimization using adaptive evolutionary algorithm with Monte Carlo tree search reinforcement learning. *Energy Convers. Manag.* **252**, 115047 (2022). <https://doi.org/10.1016/j.enconman.2021.115047>
61. G. Mosetti, C. Poloni, B. Diviacco, Optimization of wind turbine positioning in large windfarms by means of a genetic algorithm. *J. Wind Eng. Ind. Aerodyn.* **51**(1), 105–116 (1994). [https://doi.org/10.1016/0167-6105\(94\)90080-9](https://doi.org/10.1016/0167-6105(94)90080-9)
62. S. Grady, M. Hussaini, M.M. Abdullah, Placement of wind turbines using genetic algorithms. *Renew. Energy* **30**(2), 259–270 (2005). <https://doi.org/10.1016/j.renene.2004.05.007>
63. X. Gao, H. Yang, L. Lin, P. Koo, Wind turbine layout optimization using multi-population genetic algorithm and a case study in Hong Kong offshore. *J. Wind Eng. Ind. Aerodyn.* **139**, 89–99 (2015). <https://doi.org/10.1016/j.jweia.2015.01.018>

64. A.M. Abdelsalam, M. El-Shorbagy, Optimization of wind turbines siting in a wind farm using genetic algorithm based local search. *Renew. Energy* **123**, 748–755 (2018). <https://doi.org/10.1016/j.renene.2018.02.083>
65. R. Shakoor, M.Y. Hassan, A. Raheem, Y.-K. Wu, Wake effect modeling: a review of wind farm layout optimization using Jensen's model. *Renew. Sustain. Energy Rev.* **58**, 1048–1059 (2016). <https://doi.org/10.1016/j.rser.2015.12.229>
66. C. Wan, J. Wang, G. Yang, H. Gu, X. Zhang, Wind farm micro-siting by gaussian particle swarm optimization with local search strategy. *Renew. Energy* **48**, 276–286 (2012). <https://doi.org/10.1016/j.renene.2012.04.052>
67. Y. Eroğlu, S.U. Seçkiner, Design of wind farm layout using ant colony algorithm. *Renew. Energy* **44**, 53–62 (2012). <https://doi.org/10.1016/j.renene.2011.12.013>
68. F.G. Montoya, F. Manzano-Agugliaro, S. López-Márquez, Q. Hernández-Escobedo, C. Gil, Wind turbine selection for wind farm layout using multi-objective evolutionary algorithms. *Expert Syst. Appl.* **41**(15), 6585–6595 (2014). <https://doi.org/10.1016/j.eswa.2014.04.044>
69. H. Yang, S. Gao, Z. Lei, J. Li, Y. Yu, Y. Wang, An improved spherical evolution with enhanced exploration capabilities to address wind farm layout optimization problem. *Eng. Appl. Artif. Intell.* **123**, 106198 (2023). <https://doi.org/10.1016/j.engappai.2023.106198>
70. T. Kunakote, N. Sabangban, S. Kumar, G.G. Tejani, N. Panagant, N. Pholdee, S. Bureerat, A.R. Yildiz, Comparative performance of twelve metaheuristics for wind farm layout optimisation. *Arch. Comput. Methods Eng.* **29**, 717–730 (2022). <https://doi.org/10.1007/s11831-021-09586-7>
71. Z. Xu, Z. Wang, J. Li, T. Jin, X. Meng, S. Gao, Dendritic neuron model trained by information feedback-enhanced differential evolution algorithm for classification. *Knowl. Based Syst.* **233**, 107536 (2021). <https://doi.org/10.1016/j.knosys.2021.107536>
72. S. Chen, J. Montgomery, A. Bolufé-Röhler, Measuring the curse of dimensionality and its effects on particle swarm optimization and differential evolution. *Appl. Intell.* **42**, 514–526 (2015). <https://doi.org/10.1007/s10489-014-0613-2>
73. D. Tang, Spherical evolution for solving continuous optimization problems. *Appl. Soft Comput.* **81**, 105499 (2019). <https://doi.org/10.1016/j.asoc.2019.105499>
74. J. Tian, Y. Lu, X. Zuo, Y. Liu, B. Qiao, M. Fan, Q. Ge, S. Fan, A novel image encryption algorithm using pwlcmm map-based cml chaotic system and dynamic dna encryption. *Multimed. Tools Appl.* **80**(21), 32841–32861 (2021). <https://doi.org/10.1007/s11042-021-11218-y>
75. A. Hasheminejad, M. Rostami, A novel bit level multiphase algorithm for image encryption based on pwlcmm chaotic map. *Optik* **184**, 205–213 (2019). <https://doi.org/10.1016/j.jleo.2019.03.065>
76. A. Baranovsky, D. Daems, Design of one-dimensional chaotic maps with prescribed statistical properties. *Int. J. Bifurc. Chaos* **5**(06), 1585–1598 (1995). <https://doi.org/10.1142/S0218127495001198>
77. X. Ju, F. Liu, L. Wang, W.-J. Lee, Wind farm layout optimization based on support vector regression guided genetic algorithm with consideration of participation among landowners. *Energy Convers. Manag.* **196**, 1267–1281 (2019). <https://doi.org/10.1016/j.enconman.2019.06.082>
78. UCI Machine Learning Repository (2023). <https://archive.ics.uci.edu/>
79. J.F. Khaw, B. Lim, L.E. Lim, Optimal design of neural networks using the Taguchi method. *Neurocomputing* **7**(3), 225–245 (1995). [https://doi.org/10.1016/0925-2312\(94\)00013-1](https://doi.org/10.1016/0925-2312(94)00013-1)
80. S. García, D. Molina, M. Lozano, F. Herrera, A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: a case study on the cec'2005 special session on real parameter optimization. *J. Heuristics* **15**, 617–644 (2009). <https://doi.org/10.1007/s10732-008-9080-4>
81. S. Holm, A simple sequentially rejective multiple test procedure. *Scand. J. Stat.* **6**, 65–70 (1979)
82. D. Groppe, *Bonferroni-Holm Correction for Multiple Comparisons*. MATLAB Central File Exchange (2025). <https://www.mathworks.com/matlabcentral/fileexchange/28303-bonferroni-holm-correction-for-multiple-comparisons>
83. G.E. Noether, *Introduction to Wilcoxon (1945) Individual Comparisons by Ranking Methods* (Springer, New York, 1992), pp. 191–195. https://doi.org/10.1007/978-1-4612-4380-9_15

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Haichuan Yang^{1,2} · Haotian Li³ · Yifei Yang⁴ · Naoya Chiba^{2,5} · Shingo Kagami² · Koichi Hashimoto² · Yuichi Nagata¹

✉ Haichuan Yang
yokaisen1994@gmail.com

✉ Koichi Hashimoto
koichi.hashimoto.a8@tohoku.ac.jp

✉ Yuichi Nagata
nagata@is.tokushima-u.ac.jp

Haotian Li
15916886991ht@gmail.com

Yifei Yang
yyf773906764@gmail.com

Naoya Chiba
chiba@nchiba.net

Shingo Kagami
swk@tohoku.ac.jp

¹ Graduate School of Technology, Industrial and Social Sciences, Tokushima University, Tokushima 770-8506, Japan

² Graduate School of Information Sciences, Tohoku University, Sendai 980-8579, Japan

³ Faculty of Engineering, University of Toyama, Toyama 930-8555, Japan

⁴ Faculty of Science and Technology, Hirosaki University, Hirosaki 036-8560, Japan

⁵ Cybermedia Center, Osaka University, Osaka 567-0047, Japan